INSTITUTO TECNOLÓGICO DE AERONÁUTICA



Gabriel Gandour

EXPLORING DISTRIBUTED GRAPH PROCESSING TECHNIQUES

Final Paper
2024

Course of Computer Engineering

CDU 681.3

Gabriel Gandour

EXPLORING DISTRIBUTED GRAPH PROCESSING TECHNIQUES

Advisor

Prof. Dr. Juliana de Melo Bezerra (ITA)

COMPUTER ENGINEERING

São José dos Campos
Instituto Tecnológico de Aeronáutica

2024

Cataloging-in Publication Data
Documentation and Information Division

Gandour, Gabriel
Exploring Distributed Graph Processing Techniques / Gabriel Gandour.
São José dos Campos, 2024.
76f.

Final paper (Undergraduation study) – Course of Computer Engineering– Instituto Tecnológico de Aeronáutica, 2024. Advisor: Prof. Dr. Juliana de Melo Bezerra.

1. Graphs. 2. Distributed Systems. 3. Pregel. I. Instituto Tecnológico de Aeronáutica. II. Exploring Distributed Graph Processing Techniques.

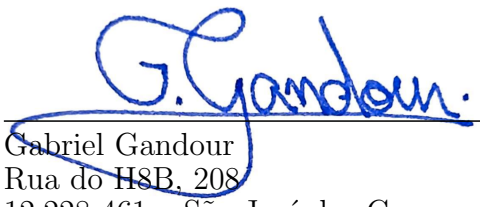
BIBLIOGRAPHIC REFERENCE

GANDOUR, Gabriel. **Exploring Distributed Graph Processing Techniques**. 2024. 76f. Final paper (Undergraduation study) – Instituto Tecnológico de Aeronáutica, São José dos Campos.

CESSION OF RIGHTS

AUTHOR'S NAME: Gabriel Gandour
PUBLICATION TITLE: Exploring Distributed Graph Processing Techniques.
PUBLICATION KIND/YEAR: Final paper (Undergraduation study) / 2024

It is granted to Instituto Tecnológico de Aeronáutica permission to reproduce copies of this final paper and to only loan or to sell copies for academic and scientific purposes. The author reserves other publication rights and no part of this final paper can be reproduced without the authorization of the author.



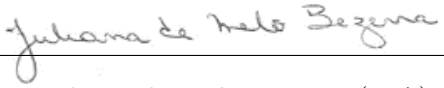
Gabriel Gandour
Rua do H8B, 208
12.228-461 – São José dos Campos–SP

EXPLORING DISTRIBUTED GRAPH PROCESSING TECHNIQUES

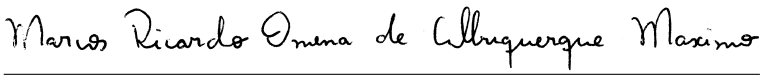
This publication was accepted like Final Work of Undergraduation Study



Gabriel Gandour
Author



Juliana de Melo Bezerra (ITA)
Advisor



Prof. Dr. Marcos Ricardo Omena de Albuquerque Maximo
Course Coordinator of Computer Engineering

São José dos Campos: November 11, 2024.

Acknowledgments

I'd like to thank my advisor, Prof. Dr. Juliana de Melo Bezerra, for her guidance and support during the development of this work.

I'm also grateful for my many friends and colleagues who made my time in college even more enjoyable.

I thank my parents, for being my first teachers and for always supporting me in my academic journey, and my brother, who has always been a great companion and an admirable friend.

Finally, I express my gratitude to Jesus Christ, my savior, who loved me before I was born. Through him all things were made; without him nothing was made that has been made.

"Gold there is, and rubies in abundance, but lips that speak knowledge are a rare jewel."
— Proverbs 20:15 NVI

Abstract

After the development of graph theory, graphs have been used to study and solve problems in many fields, such as linguistics, biology and computer science. At the same time, the amount of data gathered by companies and institutions has reached a surprisingly high level. These two facts have led to the problem of processing and analysing large graphs in a reasonable time by using distributed algorithms. The solution to write distributed algorithms for graphs in a simple and consistent way was to build tools and frameworks that simplify the process of writing and running these algorithms, such as Google's Pregel, which is a distributed graph processing framework that has been used internally by Google to solve graph problems such as page rank. This work aims to study and understand the working principles of Pregel and, inspired by it, to propose, design and implement a new framework, called Go-Pregel. Go-Pregel focuses on the educational environment, helping students to understand distributed processing concepts and learn how to solve graph problems in a distributed environment. After explaining how Go-Pregel works, we give different graph problems examples and how to solve them using this framework. Finally, we conducted a practical activity with students from a distributed processing course, and present the collected results and feedback.

Resumo

Após o desenvolvimento da teoria de grafos, estes têm sido usados para estudar e resolver problemas em muitos campos, como linguística, biologia e ciência da computação. Ao mesmo tempo, a quatidade de dados acumulada por empresas e instituições alcançou um nível surpreendentemente alto. Estes dois fatores levaram ao problema de processar e analisar grandes grafos em um tempo razoável usando algoritmos distribuídos. A solução para escrever algoritmos distribuídos para grafos de maneira simples e consistente foi construir ferramentas e *frameworks* que simplificassem o processo de escrita e execução desses algoritmos, como o Pregel, da Google, que é um *framework* de processamento de grafos distribuído usado internamente pela Google para resolver problemas como o *page rank*. Este trabalho tem como objetivo estudar e entender os princípios de funcionamento do Pregel e, inspirado por ele, propor, projetar e implementar um novo *framework*, chamado Go-Pregel. O Go-Pregel é focado no ambiente educacional, para ajudar estudantes a entender os conceitos de processamento distribuído e aprender a resolver problemas em grafos em ambientes distribuídos. Após explicar o funcionamento do Go-Pregel, são dados diferentes exemplos de problemas em grafos e suas respectivas soluções usando o *framework* desenvolvido. Finalmente, foi conduzida uma atividade prática com estudantes de um curso de processamento distribuído, e os resultados e *feedback* coletados são apresentados.

List of Figures

FIGURE 2.1 – Graph Example: Four vertices (a, b, c and d) and five edges (e_1, e_2, e_3, e_4 and e_5) 16

FIGURE 2.2 – Digraph Example: Four vertices (a, b, c and d) and five edges (e_1, e_2, e_3, e_4 and e_5) 17

FIGURE 2.3 – In the left, a graph G with four vertices, and a subgraph G' on the right. 18

FIGURE 2.4 – A digraph and its underlying undirected graph. 19

FIGURE 2.5 – Examples of different types of connected graphs. Note that the graph in (a) is the underlying undirected graph of the ones in (b) and (c). 20

FIGURE 2.6 – BSP superstep diagram (LI *et al.*, 2020) 21

FIGURE 2.7 – State machine describing the dynamics of halting and reactivating vertices 22

FIGURE 2.8 – Max Value Algorithm Example (MALEWICZ *et al.*, 2010) 23

FIGURE 3.1 – Go-Pregel architecture diagram 31

FIGURE 3.2 – Example graph image 33

FIGURE 3.3 – Master's state machine flowchart 35

FIGURE 4.1 – SSSP output generated by the visualizer tool 54

FIGURE 4.2 – Superstep progressions with and without failure. 55

FIGURE 4.3 – Connected Components output generated by the visualizer tool . . . 57

FIGURE 4.4 – Topological Sort output generated by the visualizer tool 58

FIGURE 4.5 – Bipartite graph algorithm outputs generated by the visualizer tool. . 60

FIGURE 5.1 – Answers from the feedback form 63

Contents

1 Introduction 12

1.1 Contextualization 12

1.2 Motivation 13

1.3 Objective 15

2 Background 16

2.1 Graphs 16

2.1.1 Formal Definition 16

2.1.2 Directed Graphs 17

2.2 Pregel 20

2.2.1 Bulk Synchronous Parallel 20

2.2.2 Pregel Model of Computation 21

2.2.3 Pregel Vertex Interface 23

2.2.4 Graph Partitioning 24

2.2.5 Master-Worker Architecture 25

2.2.6 Fault Tolerance 25

2.2.7 Combiners 26

2.2.8 Aggregators 26

2.3 Related Work 27

2.3.1 FLASH 27

2.3.2 Graph3S 28

2.3.3 Pregel Algorithms for Graph Connectivity Problems 28

2.3.4 Distributed Algorithms Visualization for Educational Purposes 29

3 Go-Pregel: A Distributed Graph Processing Framework . . . 30
3.1 General Architecture . . . 30
3.1.1 Storage . . . 31
3.1.2 Master Machine . . . 33
3.1.3 Worker Machine . . . 37
3.1.4 Graph Algorithm and Schema . . . 38
3.1.5 Go-Pregel Visualizer . . . 41
3.1.6 Scripts . . . 42
3.2 Go-Pregel writing guide . . . 44
4 Experiments with Go-Pregel . . . 46
4.1 Single Source Shortest Path . . . 46
4.1.1 Writing the Go-Pregel algorithm for SSSP . . . 49
4.1.2 Running the SSSP algorithm . . . 52
4.1.3 Testing fault tolerance . . . 54
4.2 Connected Components . . . 56
4.3 Topological Sort . . . 57
4.4 Graph Bipartition . . . 58
5 Educational Practice with Go-Pregel . . . 61
5.1 Pre-Lab Activity . . . 61
5.2 Activity with Go-Pregel . . . 62
5.3 Feedback form . . . 62
6 Conclusions . . . 65
6.1 Discussions . . . 65
6.2 Final Remarks . . . 67
6.3 Future Work . . . 67
Bibliography . . . 69

APPENDIX A – GO-PREGEL ALGORITHM CODES FOR THE PROBLEMS IN CHAPTER 4 71

A.1 Codes for the Connected Components problem 71

A.2 Codes for the Topological Sort problem 73

A.3 Codes for the Graph Bipartition problem 74

1 Introduction

1.1 Contextualization

In 1735, Leonard Euler solved the famous Seven Bridges of Könisberg problem. It was based on the city of Königsberg, Prussia, which was divided by the Pregel River into four landmasses, connected by seven bridges. The problem was to find a walk through the city that would cross each bridge once and only once. After Euler proved that this feat was impossible, the concept of graph theory was born, and a new field in mathematics was created. Graph Theory started as a branch of mathematics, but it has since been applied to many other fields, such as computer science, chemistry, and sociology (SHIRINIVAS *et al.*, 2010).

In summary, a graph is a data structure that consists of vertices and edges. A vertex is used to represent an entity, which can vary depending on the context, and an edge is used to represent a relationship between two (not necessarily distinct) vertices. Section 2.1.1 explains the formal definition of graph. It is a structure used to model, study and understand relationships between objects, and it's particularly useful when handling two types of tasks.

The first one is data visualization. Graphs can be used to represent data in a way that is easy to understand and interpret. Substituting big tables and spreadsheets with long lists of relationships by a graph can make it easier to find patterns and insights (MICHAILIDIS; LEEUW, 2001). Graph visualization is used, for example, in geographical maps: roads are edges that connect different places (vertices). Another popular concept is the Knowledge Graph. The vertexes of a single knowledge graph represent different types of entities, such as people, places and objects, and the edges represent different relationships between these entities, such as "is a friend of", "lives in" or "possesses" (PENG *et al.*, 2023).

The second type of task is decision making and problem solving (LI *et al.*, 2022). This is the topic this work focuses on. It involves taking a problem, representing it through graphs, and then using algorithms to find a solution, and deciding on an action to take depending on the output of the algorithm. The problems that can be solved using graphs are diverse. A famous example is the Global Positioning System (GPS), which proposes to

trace a route that takes a person from one place to another, preferably using the shortest distance or shortest time possible. This problem has a very famous solution in graph theory, called the Dijkstra algorithm, which finds the shortest path between two vertices in a graph. In this case, each location in the map is a vertex, and each street, avenue, road, roundabouts and other connecting elements are edges that go from one vertex to another. Another famous, but not so obvious example solvable by graphs is Sudoku. Sudoku is a game where the player must fill a 9x9 grid with numbers from 1 to 9, in such a way that each row, each column and each 3x3 subgrid contains all the numbers from 1 to 9. This problem can be represented as a graph, where each cell in the grid is a vertex, and each edge represents a constraint between two cells. When represented this way, the problem gets reduced into a graph coloring problem, another famous problem in graph theory.

1.2 Motivation

The amount of data available nowadays is overwhelming and growing fast, and data-related companies have to know how to deal with all this information. There are two well-known problems that represent this very well. The first one, the Page Rank problem (PAGE *et al.*, 1998), is one that Google has solved and has to do with its most famous product: the search engine. When a user types a query in Google's search bar, in which order should the results be shown? In other words, how should the pages be ranked? The internet has more than 5 billion pages, and yet Google ranks the results efficiently in less than a second. One of the most important steps is to determine the most popular websites by comparing how many pages links to a certain result. If many pages links to it, it is probably a good result. This algorithm is run by considering the internet as a graph, where each page is a vertex and each link is an edge. The challenge in this problem is to execute such computation in a large graph in a reasonable amount of time.

Another problem is a problem faced by Facebook and other social media: the Community Detection problem (FORTUNATO; HRIC, 2016). Given all the users and friendship relationships between users, who should be recommended as a potential friend to whom? In other words, which group of people have the greatest probability of being friends in real life? Finally, which communities will a user probably like to participate in? These are questions answered by solving this problem. This problem can also be interpreted by using a graph, where each user is a vertex and each friendship relationship is an edge.

Having to solve these problems leads to two concerns about large scale graph problems: distributed storage, as the graphs in these situations probably won't fit in the storage of a single machine, and distributed processing. Running an algorithm for a large number of vertices in a graph, even for a simple algorithm, can be very time consuming, and the

machine's memory might not be sufficient to store the entire graph. This is why distributed processing is so important. Splitting the computation across multiple machines can speed up the processing time, give a faster result and lower the amount of memory necessary in a machine to process the graph. This work focuses on the second concern: distributed processing.

Because of these kind of problems, it has become necessary to have frameworks and tools that allow us to write distributed algorithms for graphs. Different groups of scientists and engineers have tried to come up with Distributed Graph Processing models, frameworks and tools to help people write and run these algorithms. Many solutions appeared such as PowerGraph (GONZALEZ *et al.*, 2012), GraphX (GONZALEZ *et al.*, 2014) and Gemini (ZHU *et al.*, 2016), but two of the most famous ones are Pregel (MALEWICZ *et al.*, 2010) and Apache Giraph (APACHE..., 2020). Apache Giraph was the solution presented by Meta to solve the problems related to Facebook's huge social graph (CHING *et al.*, 2015). It's an open source solution that uses Apache Hadoop under the hood to distribute the work over worker machines.

Pregel, on the other hand, is the solution presented by Google to aid distributed graph processing. Although it is not an open source solution, it has a published paper by Google explaining in high level how Pregel works.

All those tools work in a different way, but most of them try to show the same three main features: scalability, ease to use and fault tolerance (HEIDARI *et al.*, 2018). Scalability is the ability to run the algorithm in an arbitrary number of machines (after all, the graph can be arbitrarily large). Ease to use is necessary because the objective is that the user writes the code worrying more about the algorithm logic than about the distributed aspect of the computation. Fault tolerance is the ability to recover from a failure in one of the machines and continue the computation without losing the results. However, these tools are often used as black-boxes and the user doesn't know how they work internally.

For that reason, this work proposes to understand Pregel and study how it works. Pregel has three main advantages that makes it a good choice for this study: it is built and used by Google, a very high-reputation company, so it is a popular framework. Furthermore, it has an official paper by Google explaining how it works, at least in a high level. Finally, it is a technology that doesn't depend on any other frameworks. It is a standalone solution, which makes it easier to understand. Understanding this technology means learning how the machines distribute work among them, how fault tolerance is implemented, how the user writes the distributed version of an algorithm in Pregel, and how we can debug and test a distributed graph algorithm.

1.3 Objective

This work aims to design and implement a framework that aids the distributed processing of graph algorithms. The goal is to apply this to the educational environment, to help students uncover distributed processing concepts and learn how to adapt sequential algorithms into their distributed versions. This framework will be called "Go-Pregel". For this to be accomplished, some requirements must be established:

- The framework will be build using Google's Pregel as a reference;
- The framework must be algorithm-agnostic and easy to use (this means that the framework won't be designed to solve a specific problem, but to be able to solve any problem inside the proposed scope);
- The primary language used will be GoLang. Golang is a language known for its efficiency and have many concurrency features implemented natively, besides having grown in popularity in the last years;
- Docker will be used to containerize the framework and allow it to be run in different environments;
- A simple UI will be provided for users to understand the intermediate steps of the distributed algorithm execution. This will also help students to debug their algorithms.

There is also two assumptions that will be considered during the development of this framework: the first is that the work will be restricted to static directed graphs. Static means that there no vertices will be created nor deleted during the execution of the algorithms (*i.e.* the topology of the graph doesn't change), and directed that the edges will be directed, that is, if vertex A is connected to vertex B, B isn't necessarily connected to A. This concept will be properly defined in Section 2.1.2. The second assumption is that the vertices and edges may or may not have associated data depending on the problem to be solved. The structure and schema of this associated data will be defined by the user. An edge, for example may have a weight (an integer) and a vertex could have a color (a string). We'll call these pieces of data "edge values" and "vertex values".

2 Background

In this chapter, we present the necessary background and technical concepts to understand and implement the proposed framework.

2.1 Graphs

Graphs are a data structure that represents a set of objects and the relationships between them. The objects are called vertices (the plural of vertex) and the relationships are called edges. To understand the next sections, it is necessary to stablish some formal definitions about graphs and its properties.

2.1.1 Formal Definition

A graph G compreends a non-empty finite set of vertices denoted by $V(G)$ and a finite set of edges $E(G)$ (WEST, 2001). An edge is a relationship between two vertices (not necessarily distinct) called *endpoints* of the edge. When there is an edge between two vertices u and v, we say that u and v are *connected*, *adjacent* or *neighbors*.

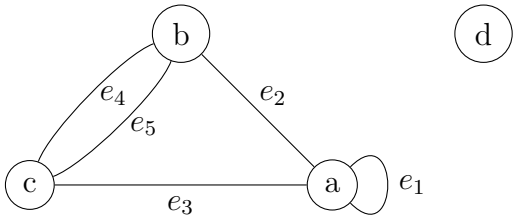


FIGURE 2.1 – Graph Example: Four vertices (a, b, c and d) and five edges (e_1, e_2, e_3, e_4 and e_5)

As shown in the example in Figure 2.1, a vertex can be connected to itself (vertex a) or to others. A vertex can be connected to any number of vertices (including zero). When an edge connects a vertex to itself (edge e_1), it is called a *loop*. Note that it is also possible for two edges to share the same endpoints.

2.1.2 Directed Graphs

Similar to a graph, a directed graph (or a digraph) also consists of a non-empty finite set of vertices denoted by $V(G)$ and a finite set of edges $E(G)$ (WILSON, 1996). The difference here is that an edge is an ordered pair of vertices (u, v), where the first vertex is the *source* and the second is the *target* of the edge. If there is an edge from u to v, we say that v is an out-neighbor of u, and u is an in-neighbor of v. In this work, in a digraph, u is said to be a *neighbor* of v (without the prefix) if and only if there are edges (u, v) and (v, u). Furthermore, an edge (u, v) is said to be an out-edge of u and an in-edge of v. The visual representation of a digraph is also similar to the representation of a graph, but the edges are drawn as arrows, to indicate the direction of the edge. In Figure 2.2, we have an example of a digraph.

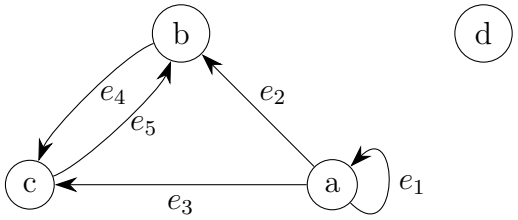


FIGURE 2.2 – Digraph Example: Four vertices (a, b, c and d) and five edges (e_1, e_2, e_3, e_4 and e_5)

In Figure 2.2, c is an out-neighbor of a, a is an in-neighbor of c and b is a neighbor of c. The out-edges of a are e_1, e_2 and e_3, and the in-edges of c are e_3 and e_4.

Another important definition that will be used in the next sections is the *degree* of a vertex. The degree of a vertex u, noted by $d(u)$, is the number of out-edges of u. In Figure 2.2, $d(a) = 3$, $d(b) = 1$, $d(c) = 1$ and $d(d) = 0$.

Both Pregel and Go-Pregel work with directed graphs. For simplicity, we'll refer to them simply as graphs in the next sections.

2.1.2.1 Subgraphs

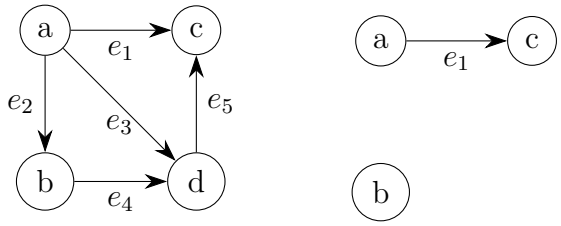
A graph G' is defined as a subgraph of another graph G if and only if three conditions are met:

1. $V(G') \subset V(G)$;
2. $E(G') \subset E(G)$;
3. $(u, v) \in E(G')$ implies $u, v \in V(G')$;

Additionally, a graph partition has a similar definition to subgraphs, with a change in the third condition. A graph G' is a partition of another graph G if and only if:

1. $V(G') \subset V(G)$;
2. $E(G') \subset E(G)$;
3. $(u, v) \in E(G) \wedge u \in V(G')$ implies $(u, v) \in E(G')$;

Note that a graph partition isn't actually a graph, since it is possible for an edge to point to a vertex outside the partition. Nevertheless, a partition is still a set of vertices and edges, and this definition will be useful to explain how Pregel works. Figure 2.3 shows an example of a graph and a subgraph. Note that, eventhough e_2 is not in G', G' is still a subgraph of G.



(a) Graph G (b) Graph G'

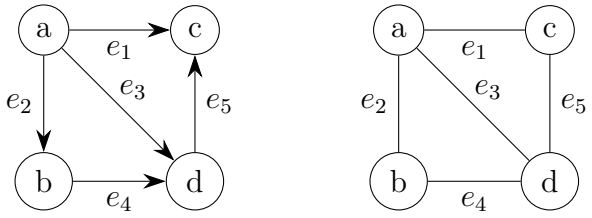
FIGURE 2.3 – In the left, a graph G with four vertices, and a subgraph G' on the right.

2.1.2.2 Underlying Undirected Graphs

Another useful definition is the *underlying undirected graph* of a directed graph. The underlying undirected graph of a directed graph G is an undirected graph G' such that:

1. $V(G') = V(G)$;
2. For every edge $(u, v) \in E(G)$, there is an $e \in E(G')$ that connects u and v in G';
3. Every edge in G' is generated by the previous item.

We'll denote the underlying undirected graph of a graph G as $U(G)$. Figure 2.4 shows an example of a directed graph and its underlying undirected graph.



(a) A digraph G

(b) G's underlying undirected graph $G' = U(G)$

FIGURE 2.4 – A digraph and its underlying undirected graph.

2.1.2.3 Connected Components

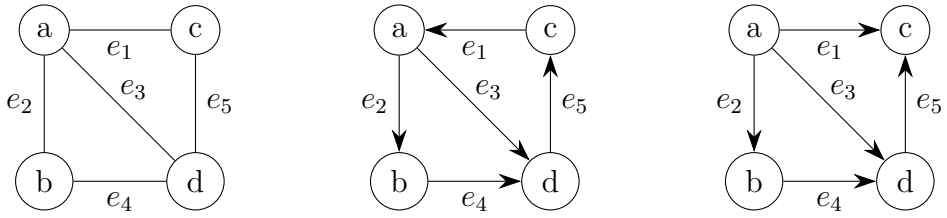
A path in a graph is a sequence of edges $p = (e_1, e_2, \ldots, e_n)$ such that the target of e_i is the source of e_{i+1} for all $i \in \{1, 2, \ldots, n-1\}$. The definition of path leads to another very important definition: the connected components of a graph.

In an undirected graph G, a connected component C of G is a subgraph of G such that, for every pair of vertices $u, v \in V(C)$, there is a path connecting u and v. In other words, a connected component is a subgraph where every pair of vertices is connected by a path. If an undirected graph is a connected component of itself, it is said to be *connected*.

In a directed graph, the definition requires a subtle change. A strongly connected component C of a directed graph G is a subgraph of G such that, for every pair of vertices $u, v \in V(C)$, there is a path from u to v and a path from v to u. This means that, in a directed graph, a connected component is a subgraph where every pair of vertices is connected by a path in both directions. If a digraph is a strongly connected component of itself, it is said to be *strongly connected*.

On the other hand, a subgraph W of a graph G is a weakly connected component if and only if $U(W)$ is a connected component of $U(G)$. In other words, a weakly connected component is a subgraph of a directed graph where the underlying undirected graph is a connected component. If a digraph is a weakly connected component of itself, it is said to be *weakly connected*.

Figure 2.5 shows an image with three graphs. The first is an connected undirected graph, the second is a strongly connected digraph and the third is a weakly connected digraph. For simplicity, we'll refer to weakly connected components as connected components in the next sections.



(a) A connected undirected graph

(b) A strongly connected digraph

(c) A weakly connected digraph

FIGURE 2.5 – Examples of different types of connected graphs. Note that the graph in (a) is the underlying undirected graph of the ones in (b) and (c).

2.2 Pregel

Pregel is a distributed graph processing framework developed by Google. It was first presented in the paper "Pregel: A System for Large-Scale Graph Processing" in 2010 (MALEWICZ *et al.*, 2010). The purpose of Pregel is to provide a simple and scalable way to write and run distributed graph algorithms. The input for Pregel is a graph, and the output is also a graph. As stated before in Section 1.3, the vertices and edges may have associated data, and the structure of this data is defined arbitrarily by the user. This means that the output of a Pregel algorithm will often not be the direct solution to the problem being solved, but it will be a graph in which the solution is much easier to find.

For example, if a graph G has many vertices holding integer values, and the problem is to find the maximum value among all vertices, one possible output for the Pregel algorithm would be a graph G' where each vertex holds the greatest value in the graph. The output G' itself isn't the solution to our problem, but the desired value is easily found by looking at any vertex's value in G'. The strategy for structuring the vertices and edges data depends on the problem and will sometimes require creativity from the user.

In the next sections, we'll discuss how Pregel works and how it distributes the computation across multiple machines, but some details and advanced features of the framework will be ommited.

2.2.1 Bulk Synchronous Parallel

To understand Pregel, it is necessary to know what Bulk Synchronous Parallel (BSP) (CHEATHAM *et al.*, 1994) is. BSP is a design for parallel algorithms. This means BSP is not a framework, an algorithm or a tool. It is rather a way of thinking and designing parallel algorithms. It was first proposed by Leslie Valiant in 1990. The main idea is to

divide the computation into supersteps, where each superstep is composed by three main phases: computation, communication and synchronization.

In the first phase, computation, every working machine executes a part of the algorithm until a certain point determined by the programmer. In the second phase, communication, the machines exchange messages with each other. These messages may have different purposes depending on the algorithm. Finally, the last phase, synchronization, is the point where all machines must wait for each other to finish the computation and communication phases before moving to the next superstep. This means that different machines can be on different phases, but they are always in the same superstep. This way, the algorithm is divided into a sequence of supersteps, where each superstep is composed by the three phases. Figure 2.6 (LI *et al.*, 2020) shows a diagram of the BSP model with four machines (nodes) and two supersteps.

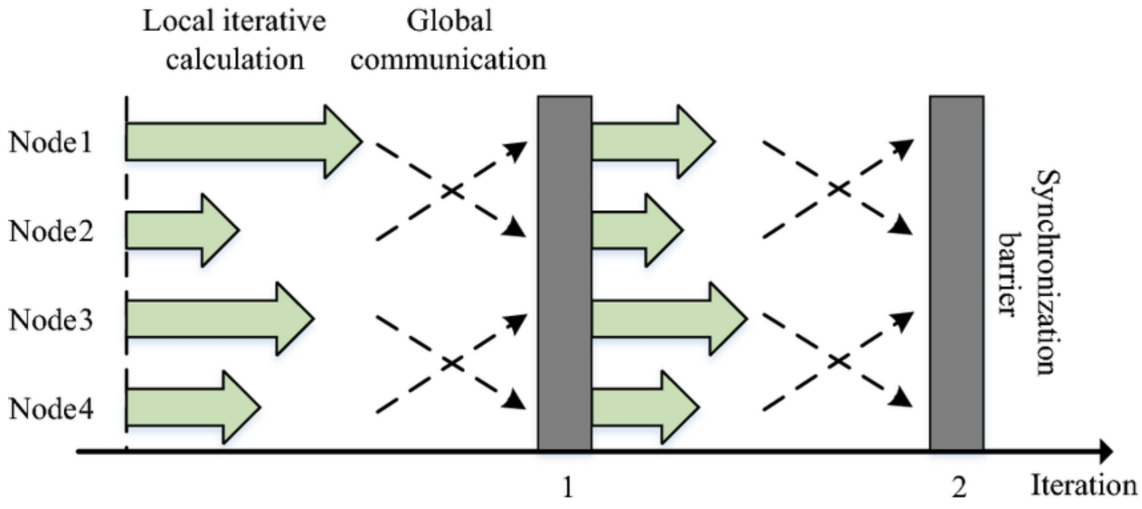


FIGURE 2.6 – BSP superstep diagram (LI *et al.*, 2020)

2.2.2 Pregel Model of Computation

Pregel interprets each vertex as an independent machine, and each vertex is responsible for its part of the computation. The framework uses the BSP model to coordinate the work. This means that the algorithm is divided into several supersteps, where each vertex executes a computation phase, communicates with other vertices and then synchronize with the other vertices. Each vertex is unable to read or change the values of other vertices, but they can read and change their own values at will. Also, each vertex, in the communication phase, can send messages to any other vertices, as long as the target vertex's ID is known. These messages do not require a reply, and they are only read in the next superstep. The decisions made in the computation phase are usually based on the messages received in the previous superstep, since they are the only way of communication

between vertices. In Google's Pregel, each vertex has also the authority to delete or create any vertex in the graph. This deletion and creation feature won't be studied in this work.

During the computation step, each vertex can vote to halt and consequently become inactive. When every vertex in the graph votes to halt, the Pregel algorithm is considered finished, and the resulting graph is written to the output. When a vertex votes to halt, it is excluded from the computation phase and stops working, unless it receives a message from another vertex. When a halted vertex receives a message, it is automatically reactivated, and its vote to halt is canceled. Deciding when a vertex should vote to halt is responsibility of the user, which will define this decision depending on the algorithm being implemented. The Figure 2.7 shows a simple state machine that shows how a vertex changes its state from active to inactive and vice-versa.

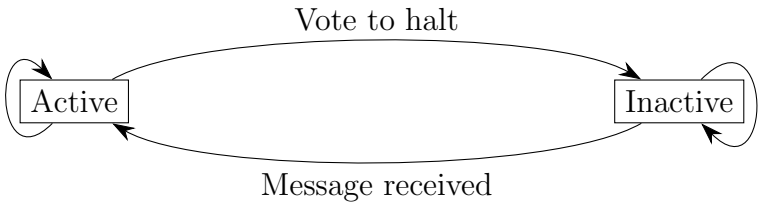


FIGURE 2.7 – State machine describing the dynamics of halting and reactivating vertices

The Figure 2.8 shows an example of a Pregel algorithm execution that finds the greatest value among all vertices in a strongly connected graph. The input of the algorithm is a strongly connected graph with four vertices and six edges (note that the bidirectional edges are actually two edges in opposite directions). Each vertex has an integer value and the goal is to find the maximum value among all vertices. The dotted arrows represent messages sent from one vertex to another. The algorithm starts with each vertex sending its value to its neighbors. In this algorithm, the message used for the communication step is simly the value that the sender currently holds. In the next superstep, each vertex receives the maximum value of its neighbors and updates its own value to the maximum value. If a vertex doesn't receive any messages in a superstep, or if all the messages have integers smaller than its own value, it votes to halt (the shaded vertices are inactive). This process repeats until all vertices vote to halt. The output of the algorithm is a graph where each vertex holds the maximum value of its neighbors. The desired value is found by looking at any vertex's value in the output graph.

In Figure 2.8 (MALEWICZ *et al.*, 2010), let A, B, C and D be the vertices from the graph, from left to right. Vertex C, for example, has the value 2 in the beginning of the algorithm, and sends a message with the value 2 to B and D in the end of superstep zero. In superstep one, C receives a message with the value 1 from D, so it doesn't change its value and votes to halt. However, in superstep 2 it receives a message from D with the

value 6. The message reactivates C, which updates its value to 6 and sends a message to B and D with the value 6. Finally, in superstep 3, C doesn't receive any messages, so it votes to halt again. The same process happens with the other vertices until all of them halt.

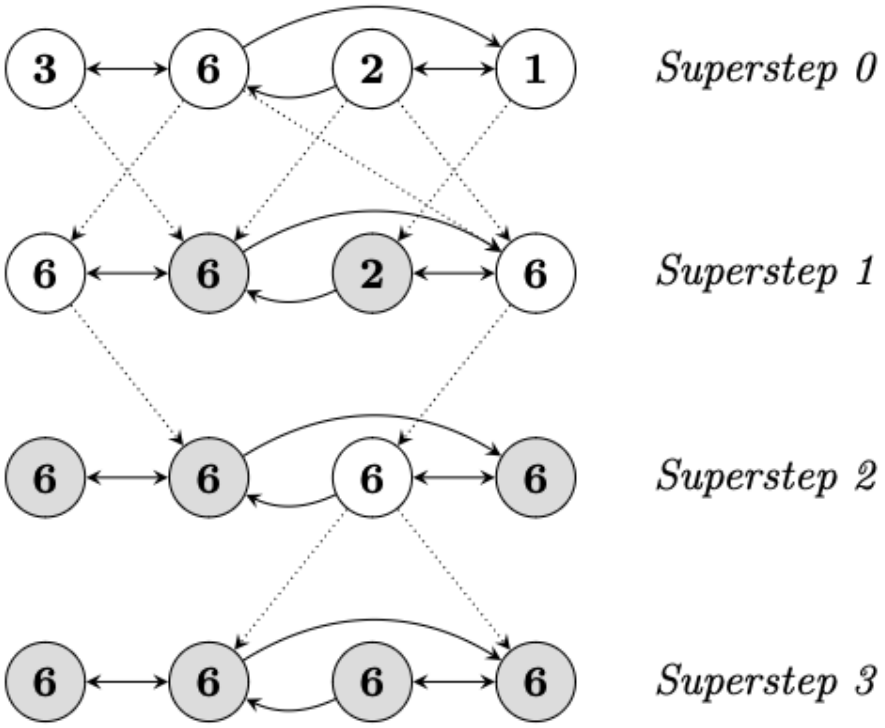


FIGURE 2.8 – Max Value Algorithm Example (MALEWICZ *et al.*, 2010)

2.2.3 Pregel Vertex Interface

In the Pregel C++ API, to implement a Pregel algorithm, the user must define a class that inherits from the `Vertex` class (provided by the framework). The user has access to the methods shown in Listing 2.1 (MALEWICZ *et al.*, 2010). The only method that the user must implement is the `Compute` method, which is called in the beginning of every superstep. This method comprehends the computation phase of the BSP model, but also the communication phase. The user has to define the rules for when and how to send messages to other vertices, using the method `SendMessageTo` to do so. The user also has to define, in the `Compute` method, the rules for when a vertex should vote to halt, using `VoteToHalt()`.

LISTING 2.1 – Pregel C++ Vertex Interface

```
template <typename VertexValue,
          typename EdgeValue,
          typename MessageValue>
class Vertex {
    public:
        virtual void Compute(MessageIterator* msgs) = 0;
        const string& vertex_id() const;
        int64 superstep() const;
        const VertexValue& GetValue();
        VertexValue* MutableValue();
        OutEdgeIterator GetOutEdgeIterator();
        void SendMessageTo(const string& dest_vertex,
                           const MessageValue& message);
        void VoteToHalt();
};
```

2.2.4 Graph Partitioning

In Pregel, each vertex is responsible for its own computation, but it doesn't mean that each machine has a single vertex. This would be impossible for large graphs that would require billions of machines. Instead, the graph is divided into several partitions (sets of vertices and their out-edges). Each partition is assigned to a machine, and the machine executes the `Compute` method for every vertex in its partition for every superstep.

This division is done in a way that the partition (and thus, the machine) a vertex is allocated to is determined only by the vertex ID and the number of partitions. This is good because locations of every vertex in the graph doesn't need to be stored in memory. On the contrary, the machines can calculate the location of a vertex in runtime.

The fact that there are multiple vertices in a single machines also means that message passing won't necessarily cause network communication. If a vertex sends a message to another vertex in the same machine, the message is sent directly, without the need to go through the network. Network communication only happens when a vertex sends a message to another vertex in a different machine.

The number of partitions in a single machine is not necessarily one. The number of partitions in a worker is defined by the master machine (to be explained in Section 2.2.5). Having more than on partition in a machine can improve the performance of the algorithm, but this won't be further discussed in this work. For the sake of simplicity, we'll assume

that each machine has only one partition of the graph.

2.2.5 Master-Worker Architecture

Pregel uses a master-worker architecture. While the workers are responsible for executing the `Compute` method for the vertices in their partition, the master is responsible for coordinating the work between all the workers. The master is responsible for distributing the partitions to the workers, but also for synchronizing the workers and ordering them to go to the next superstep. When a worker finishes the computation and communication phases, it sends a message to the master machine, indicating its superstep has ended. When all workers are ready to go on to the next superstep, the master sends a message to every worker ordering to proceed. Until then, the workers stay idle.

In short, the Pregel framework can be described with the following steps:

1. The master divides the input graph into partitions;
2. The master sends the partitions to the workers;
3. The master orders the workers to start a superstep, in which each active vertex:
   (a) checks for messages received in the previous supersteps;
   (b) in the `Compute` method, calculates their new values, if necessary;
   (c) send messages to other vertices;
   (d) decides whether to halt or not;
   (e) if every vertex halts, proceed to item 4. Otherwise, synchronize workers and go to step 3.
4. All workers write the resulting partitions to the output.

Besides organizing the partitions and synchronizing supersteps, the master also has a fundamental role in the fault tolerance of the system.

2.2.6 Fault Tolerance

In Pregel, Fault Tolerance is implemented through checkpoints. A checkpoint is a snapshot of the state of the computation at a certain superstep. When saving a checkpoint, a worker writes in a log the values of every vertex in its partition, their incoming messages, along with the respective edges and edge values. Since all the computation in a superstep

depends only on these pieces of data, it is possible to continue a Pregel algorithm from this checkpoint.

The frequency of the checkpoint saving is determined by the user, and must reach an equilibrium between I/O latency and failure risk: if the checkpoint is saved too frequently, the I/O latency will be too high, but if it is saved too rarely, the risk of losing a lot of work in case of failure is too high.

When a worker fails, the master is responsible for detecting the failure and repartitioning the graph from the last available checkpoint among the remaining workers. The master then orders the workers to continue the supersteps cycle.

2.2.7 Combiners

Sending messages from a vertex to another (especially if they are located in different machines) implies a network overhead that is not directly related to the execution of the algorithm. This overhead can be reduced by using combiners set up by the user. To minimize this overhead, the user can define a combiner function that receives a list of messages to be sent and returns an equal or shorter list of messages that would have the same effect. If, for example, a vertex sends two different messages to the same target vertex, but only the sum of the message values matter, the combiner can merge the two messages into a single one with the sum of the values. This way, there is less network overhead, and the algorithms could run faster depending on the combiner function and on the structure of the graph.

2.2.8 Aggregators

Pregel doesn't rely on shared memory, nor does it provide a function for broadcasting information. However, Pregel provides a mechanism that can be used as a form of global communication: aggregators. In every superstep S, a vertex may provide a value to the algorithm's aggregators. Pregel combines all the values provided by the vertices through a reduction function, and the result is shared to all the vertices in the next superstep. Pregel comes with a set of built-in aggregators, such as sum, minimum and maximum, but the users can define their own aggregators depending on the problem.

The algorithm for the maximum value stored in a graph shown in Section 2.2.2, for example, only works because the input graph is strongly connected. If it weren't, different vertices could have different maximum values in the end of the algorithm, making it incorrect. However, this changes if we use aggregators. The user can simply set a maximum aggregator, which would have the correct result after the first superstep. Aggregators can be used to implement simple tasks, such as determining the maximum value stored in

a graph, but they can also have much more complex use cases, such as implementing a distributed priority queue. We won't dive deeper into aggregators in this work, but they are an important feature of Pregel.

2.3 Related Work

Since large-scale graph processing has become a frequent theme in computer science, many tools have been developed to aid this kind of problem, and many studies in this field have been published. In this section, we'll discuss some relevant alternatives to pregel and other papers that helped the development of this work.

2.3.1 FLASH

A group of researches from the Alibaba group, as well as two different universities (Shangai Jiao Tong University and University of Technology in Sydney) proposed another Distributed Graph Processing framework called FLASH and exposed their ideas on a paper (LI *et al.*, 2022). In this paper, the authors argue that the best metrics to evaluate a Graph Processing tool are three: *expressiveness*, *productivity* and *efficiency*. They call them the ***EPE*** metrics.

Expressiveness measures how broad is the range of algorithms that can be implemented using the tool. *Productivity* measures how easy it is to write an algorithm using the tool. Finally, *efficiency* measures how fast the tool can run the algorithms. In the paper, they discuss how improving one of those three metrics implies on a decrease in the other two. For example, a tool that is very low-level is usually more efficient because the user can implement optimizations specific to the algorithm, but it is less productive because the user has more effort to use the tool and to think about the details of implementation.

However, they claim to have reached a good balance between the ***EPE*** metrics in FLASH framework. In their work, they measure the performance of FLASH in several algorithms, such as Breath First Serch, Closeness Centrality and Graph Coloring. After running the same tests with other popular tools, such as Pregel+, PowerGraph, and Ligra, they conclude that FLASH outperforms all the other frameworks on the ***EPE*** metrics.

Differently from FLASH, Go-Pregel is intended to be used in the educational environment, so rather than trying to obtain an equilibrium between the ***EPE*** metrics, the main focus of it will be maximizing productivity over the other two metrics. The expressiveness of the system is still important because the goal is that the users can study and implement the distributed versions of different algorithms, but it will not be the main focus of the framework.

2.3.2 Graph3S

Another work that is worth mentioning is the Graph3S framework, which is also explained in a paper by its authors (WANG *et al.*, 2020). Similarly to FLASH, the authors stablish three metrics to evaluate the Graph Processing Systems: *simplicity* (similar to FLASH's *productivity*), *efficiency* (same definition as in the previous subsection) and *scalability*, the only metric used by Graph3S that was not highlighted in FLASH.

Scalability refers to the ability of the system to run the algorithm in an arbitrary number of machines, which is important because of the large amount of data dealt by graph processing systems.

The authors also say that these three metrics are usually in conflict with each other, but they have as an objective to design a Simple, Speedy and Scalable system (hence the name Graph3S). To successfully do that, they had to sacrifice some of the expressiveness of the system: in Graph3S, each vertex can only send messages to its direct neighbors, as opposed to more popular frameworks like Pregel and Apache Giraph.

This limitation restrains it from running some algorithms that require more complex communication patterns. However, all benchmark algorithms normally used to evaluate other distributed graph processing systems can still be implemented in Graph3S, so restricting the communication in this case was considered a good trade-off.

Similarly to Graph3S, Go-Pregel will also sacrifice some expressiveness of the system, but with the main purpose of maximizing simplicity of the system. In Go-Pregel, the vertex can still send messages to any other vertices in the graph, but the deletion of vertices and edges is prohibited. Those two frameworks (Go-Pregel and Graph3S) sacrifice expressiveness in different ways to achieve different goals.

2.3.3 Pregel Algorithms for Graph Connectivity Problems

The paper Pregel Algorithms for Graph Connectivity Problems with Performance Guarantees (YAN *et al.*, 2014), differently from the previous two works, doesn't propose a new Graph Processing System, but rather studies how one should implement a Pregel algorithm to solve three graph connectivity problems: *Connected Components*, *Biconnected Components* and *Strongly Connected Components*.

In this paper, the authors define what they call "Balanced Practical Pregel Algorithms", or BPPAs: a class of pregel algorithms that obey four constraints on performance metrics. The first three constraints state that the space storage in a vertex, the computation cost in a superstep and the communication cost in a superstep have to be linear with respect to the degree of the vertex. The last one states that the algorithm

has to finish at most in a logarithmic number of supersteps with respect to the number of vertices in the graph. They also define "Practical Pregel Algorithms", or simply PPAs, which have the same definition as the BPPAs, but with slightly more relaxed constraints. Their objective is to write PPAs for the three graph connectivity problems stated and test them in real-world graphs to prove their efficiency.

To do that, they first write PPAs for smaller but known problems, such as the spanning tree problem and breadth first search, and then use these as building blocks to write the algorithms for the three graph connectivity problems.

2.3.4 Distributed Algorithms Visualization for Educational Purposes

In the paper Distributed Algorithms Visualization for Educational Purposes (KOLDEHOFE *et al.*, 1999), the authors argue that distributed algorithms are inherently difficult to teach and understand. There are many processes happening at the same time, and a large amount of data is usually involved.

Due to this complexity, the work aimed to build animations of some distributed algorithms to help students understand how these algorithms work, how they behave under different conditions, how communication happens and understand their time complexities.

They propose a tool that shows different views of the algorithm execution that evolve continuously. In other words, the tool shows in real time the state of the execution in a way that the user can understand it. This work served as an inspiration for the debugging feature of Go-Pregel, that will be mentioned in Section 3.1.6.1. Similarly to this proposed tool, the debbugging feature of Go-Pregel will generate images representing the state of the algorithm in each superstep, which can be used to understand how the algorithms behaves over time.

3 Go-Pregel: A Distributed Graph Processing Framework

This chapter aims to present the design for Go-Pregel, the distributed graph processing framework developed in this work. As stated in Section 1.3, the framework was built using Google's Pregel as a reference, but was programmed in GoLang (thus the name Go-Pregel). The project's repository is available at https://github.com/GaGandour/Go-Pregel (GANDOUR, 2024).

3.1 General Architecture

In this section, we'll study the general architecture of the Go-Pregel framework, its components and how they interact with each other. We'll start by learning how the graphs are stored, and then dive into the different components of Go-Pregel framework itself. Finally, we'll describe briefly the functionalities of the Go-Pregel Visualizer, and the scripts provided to easily operate the framework. Each component is shown in the diagram in Figure 3.1.

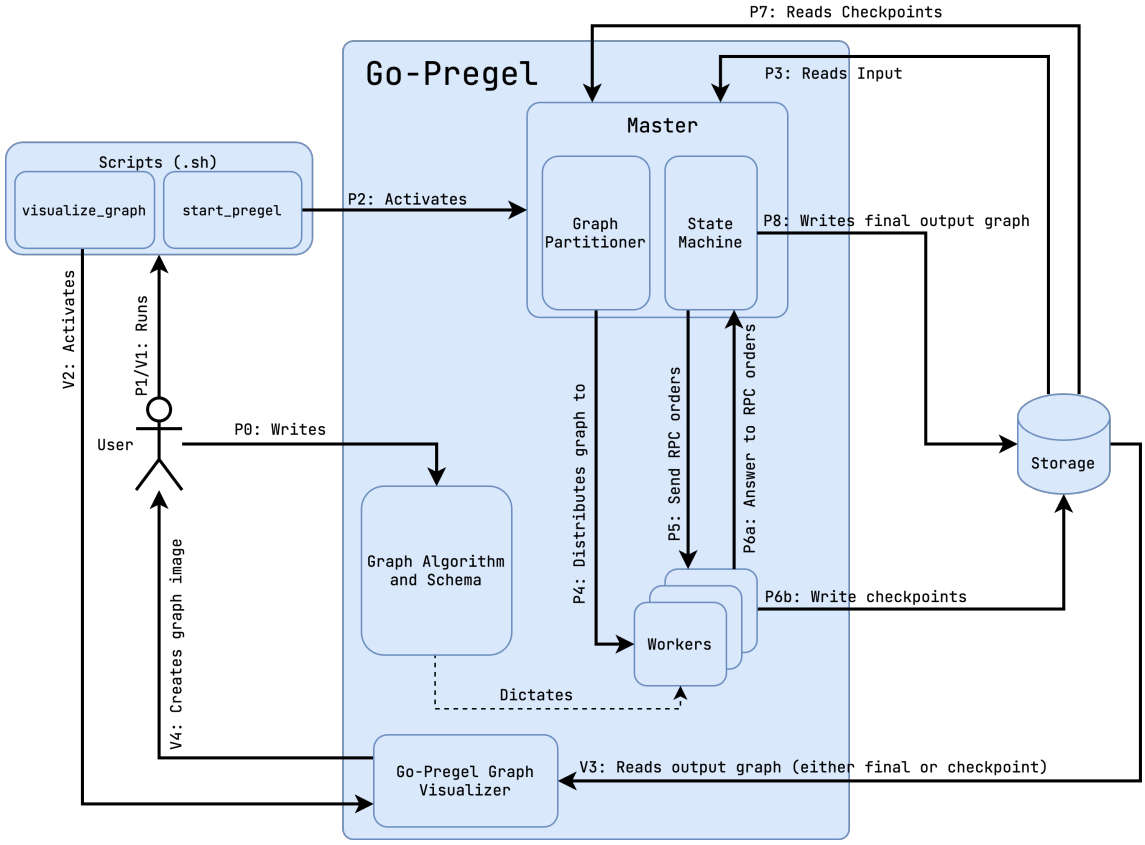


FIGURE 3.1 – Go-Pregel architecture diagram

3.1.1 Storage

As stated in Section 2.2, the input and the output of any Pregel algorithm are graphs. Since we won't focus on distributed storage, the graphs will be stored as a *JSON* file in the local file system. This means that, in order to run an algorithm in a graph, the user should provide it in the *JSON* format, and Go-Pregel will take the provided file as an input. The project's repository already comes with a few graph files generated for a set of four algorithms, but the user is free to create their own.

The provided graph files are all stored in the `graphs/` directory, and are separated by subdirectories named after a graph problem to be solved. The graph files provided to test a Single Source Shortest Path algorithm (refer to Section 4.1), for example, are stored under `graphs/sssp/`. Every graph file the user provides also has to be stored in a subdirectory of the `graphs/` directory. Along with the graph files, the repository also comes with the expected output files for each algorithm, under the `answers/` folder. These files are stored in the same structure as the input files, meaning that the expected output for a `graphs/sssp/graph.json` file is stored in the `answers/sssp/graph.json` file.

A graph json follows a specific schema, as shown in Figure 3.1. The `Vertexes` field is a map of vertexes, where each vertex has an `Edges` field, which is a map of edges. Each edge has a `To` field, which is the target vertex id. The `Value` field is a placeholder for the vertex and edge values, and the schema for these values are defined by the user, (to be specified in Section 3.1.4.2). The *Value* fields are optional and its schema (if not empty) depends on the problem to be solved. The Figure 3.2 shows an example of a graph file, and the Figure 3.2 shows the graph image representation.

After Go-Pregel is run, the output graph is stored under the `src/output_graphs/`, with the same structure as in the `graphs/` directory. This means that, if the user runs Go-Pregel for the graph file `graphs/sssp/graph.json`, the output graph will be stored in the file `src/output_graphs/sssp/graph.json`.

LISTING 3.1 – Input Graph Json Format

```
{
    "Vertexes": {
        "<vertex id>": {
            "Value": {
                // Vertex Value Schema
            },
            "Edges": {
                "<edge id>": {
                    "To": "<target vertex id>",
                    "Value": {
                        // Edge Value Schema
                    }
                }
            }
        }
    }
}
```

LISTING 3.2 – Example graph file

```
{
    "Vertexes": {
        "1": {
            "Edges": {
                "12": {
                    "To": "2"
                }
            }
        },
        "2": {
```

```
            "Edges": {}
        },
        "3": {
            "Edges": {
                "34": {
                    "To": "4"
                }
            }
        },
        "4": {
            "Edges": {}
        }
    }
}
```

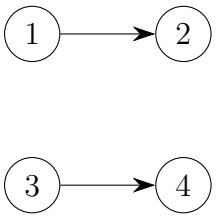


FIGURE 3.2 – Example graph image

3.1.2 Master Machine

As already stated in Section 2.2.5, the Go-Pregel framework uses a master-worker architecture, and the master and workers communicate via RPC (Remote Procedure Call). The GoLang standard library provides a package for RPC (`"net/rpc"`), which was used in this work. The master machine in Go-Pregel has three main responsabilities. The first is to partition the graph among the workers, the second is to orchestrate the supersteps, and the third is to merge the output files generated by the workers. These three responsabilities are all managed by the Pregel State Machine, which is described in Section 3.1.2.2.

3.1.2.1 Graph Partitioning

The graph partitioning is a simple process in Go-Pregel, and it is the first thing done by the master machine after establishing the connection with the workers. Graph Partitioning is the act of dividing the graph into N partitions, and assigning each partition to a worker. After establishing the master-worker connections, each worker is assigned an integer ID based on the order they connected to the master. The ID assignment begins at 0 and increments by 1 for each worker.

The master then reads the input graph from the storage and divides it among the workers following the pseudo-code in Figure 3.3. The master sends the partitions to the workers through an RPC call, and the workers store the them in their local memory. The master decides if a vertex belongs to a worker by computing the partition a vertex should be in, and comparing it to the worker's ID. The Golang function for the computation of the partition ID is shown in Figure 3.4. Note that the partition ID from a vertex depends only on its ID and on the number of partitions.

LISTING 3.3 – Graph partitioning pseudo-code

```
for worker in workers {
    partition = {} // empty partition
    for vertex in graph {
        if vertex belongs to worker {
            partition.add(vertex)
        }
    }
    send partition to worker
}
```

LISTING 3.4 – Vertex's partition computing

```
func GetPartitionIdFromVertex(
    numberOfPartitions int,
    vertexId           VertexIdType,
) int {
    return customHash(vertexId) % numberOfPartitions
}
```

3.1.2.2 Master's State Machine

The superstep orchestration logic is a big state machine with different actions and conditions. This state machine is represented by the flowchart in Figure 3.3, which is explained in the following list of nodes.

A **Beginning:** The initial state of the state machine. Nothing is done in this state, but when this machine is initialized, the connections between the master and workers are already established. The state machine then goes to the ***B*** state.

B **Read input graph:** The master reads the input graph from the storage, and goes to the ***C*** state.

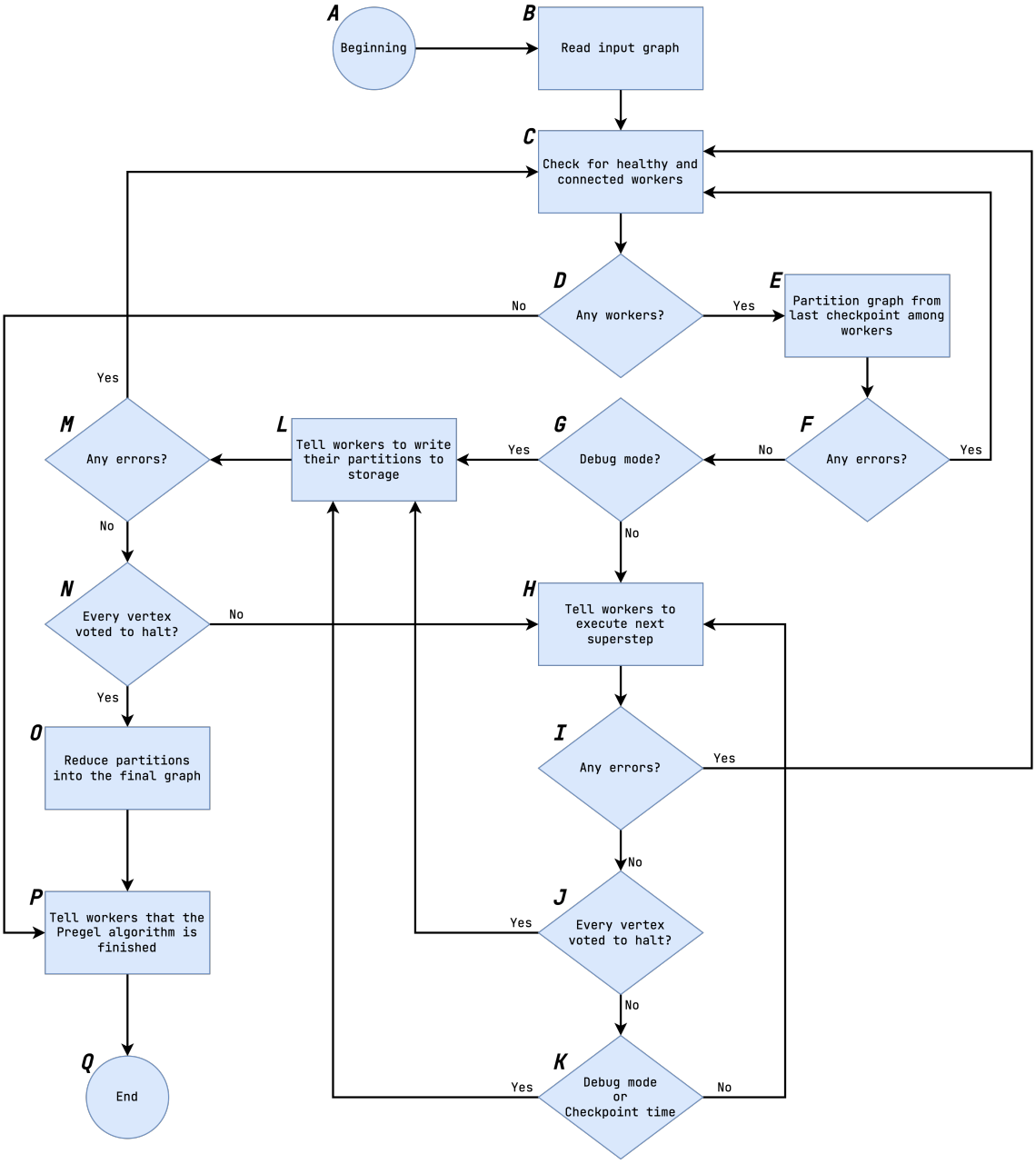


FIGURE 3.3 – Master's state machine flowchart

C **Check for healthy and connected workers:** The master sends heartbeat messages to the workers and waits for their replies to check if they are still healthy and connected. The logic then goes to the ***D*** decision.

D **Are there any healthy workers?** If there are still healthy workers after ***C*** (which is the most common scenario), the flow goes to the ***E*** state. Otherwise, the master goes to the ***P*** state.

E **Partition graph from last checkpoint:** Here, the master master checks if there is a checkpoint file for the current algorithm. These checkpoint files can be created

at the state ***L***. If there is a checkpoint file, the master reads it and partitions the graph from the last checkpoint. Otherwise, the master partitions the graph from the input file, which was read at state ***B***. The master then sends the partitions to the workers and goes to the ***F*** decision.

F **Were there any errors?** The master waits for all workers to reply their statuses from state ***E***. If there were any errors, the master goes back to the ***C*** state. Otherwise, the master goes to the ***G*** state.

G **Is Go-Pregel in debug mode?** Programming in Go-Pregel can be rather conter-intuitive at first, so we provide a debug mode to help the users understand the impact of their code on the algorithm. The debug mode is explained in Section 3.1.6.1 If the debug mode is enabled, the state machine goest to the ***L*** state. Otherwise, it goes to the ***H*** state.

H **Tell workers to execute next superstep:** The master sends a message to all workers to start the superstep. The master then goes to the ***I*** decision.

I **Were there any errors?** This is similar to decision ***F***. If there was any error, the flow goes back to state ***C***. Otherwise, it goes to the ***J*** decision.

J **Has every vertex voted to halt?** After executing superstep, one of the pieces of information that the workers send back to master as a reply to the RPC request is a boolean indicating if every vertex in their partition has voted to halt. If every vertex from every worker has voted to halt, the flow goes to state ***L***. Otherwise, it goes to another decision node ***K***.

K **Are we in debug mode? Is it checkpoint time?** If the debug mode is enabled, the master goes to state ***L***. Otherwise, the master checks if it is time to create a checkpoint. This is decided based on the current superstep number and on a checkpointing interval set by the user. If it is, the master goes to state ***L***. Otherwise, it goes back to state ***H***, which initiates another superstep.

L **Tell workers to write their partitions to storage:** The master sends a message to all workers to write their partitions to the storage. This partition file has to be usable as a checkpoint, so it has to contain the current values stored in the vertices, edges, and the messages to be sent to other vertices. The master then goes to the ***M*** decision.

M **Were there any errors?** Similar to decision ***F***. If there was any error, the flow goes back to state ***C***. Otherwise, it goes to the ***N*** decision.

N **Has every vertex voted to halt?** In this decision node, the master recalls the replies received from the workers in the last superstep. If every vertex from every worker has voted to halt, the flow goes to state ***O***. Otherwise, it goes back to state ***H***, which initiates another superstep.

O **Merge partitions and write output graph:** The master reads the partitions written by the workers and merges them into a single file. This file is the output graph, and it is written to the storage. The master then goes to the ***P*** state.

P **Tell workers that the Pregel algorithm has finished:** The master sends a message to all workers to finish the computation, and they all close their connections and turn themselves off. The master then goes to the ***Q*** state.

Q **End:** The final state of the state machine. The master has finished the computation and the output graph is ready to be read by the user.

3.1.3 Worker Machine

The worker machines, on the other hand, have simpler responsabilities. The workers have four different functions that are called by the master via RPC calls, and each of these functions are called depending on the master's current state machine node. The most important of these functions is the `RunSuperStep` function, which is expressed by the simplified pseudo-code in Figure 3.5. This function loops over the vertices in the worker's partition and executes the superstep for each vertex. The superstep of a vertex consists on two steps: the first one is to check if the vertex is active or if it has incoming messages. The latter condition is important because a vertex is reactivated if it has incoming messages. If both conditions are true, the vertex executes its `SuperStep` method. After iterating through all vertices in the subgraph, the worker compiles all messages that are to be sent from the vertices to other vertices, and send them to the apropriate workers. Finally, the function checks if every vertex has voted to halt, and sends a reply to the master with this information. It's important to note that, since a message reactivates a vertex, if any messages were sent in the superstep, the worker will consider that not every vertex has voted to halt.

LISTING 3.5 – Simplified pseudo-code of the `RunSuperStep`.

```
1 RunSuperStep() {
2     for vertex in subgraph {
3         if vertex is active or vertex has incoming messages {
4             vertex.Superstep()
5         }
6     }
```

```
7       SendMessages()
8       every_vertex_voted_to_halt = CheckIfEveryVertexVotedToHalt()
9       SendReplyToMaster(every_vertex_voted_to_halt)
10  }
```

3.1.3.1 Message Combining

As explained in Section 2.2.7, sending messages from one vertex to another in a different partition may lead to a network overhead. In order to reduce the amound of messages sent, Go-Pregel uses the `CombinePregelMessages` function to group messages addressed to the same vertex. This function receives a list of messages and returns another list of messages.

Before effectively sending the messages in Figure 3.5, Go-Pregel groups the messages addressed to the same vertex and calls the `CombinePregelMessages` function on these groups. Only the resulting message list is effectively sent to the target vertex. The criteria and logic to combine messages depend on the user's algorithm, and thus, the `CombinePregelMessages` function must be implemented by the user. If the user does not implement this function, the function is set to return the same list of messages passed as an argument. This way, implementing the combiner is optional and not doing so won't affect the algorithm's correctness.

3.1.4 Graph Algorithm and Schema

The graph algorithm executed by Go-Pregel is programmed by the user by changing two files in the project. However, to understand how the user operates the framework and which files are to be modified, we need to understand how a vertex's superstep is executed.

3.1.4.1 Vertex Superstep

The vertex superstep also follows a simple logic, written again in pseudo-code in Figure 3.6. If the vertex received messages from the last superstep, it is activated. This only has effect if the vertex voted to halt in a previous superstep and is currently inactive. Then, if the vertex is still inactive, the superstep stops immediately. Otherwise, the vertex proceeds to execute either the `ComputeInSuperStepZero` method if it is the first superstep, or the `Compute` method otherwise. These methods are defined by the user and are the ones that determine the logic of the algorithm to be run on the input graph. We decided to split the compute method into two because some algorithms have different logic

in the first superstep, so this avoids the need for a conditional in the user's code. However, if it is not the case, the user doesn't need to implement the `ComputeInSuperStepZero` method, as it copies the `Compute` method's behaviour as default.

LISTING 3.6 – Simplified pseudo-code of the `vertex.SuperStep` method.

```
(vertex) SuperStep() {
    if vertex received messages {
        vertex.Activate()
    }
    if vertex is not active {
        return
    }
    if superstepNumber == 0 {
        vertex.ComputeInSuperStepZero()
    } else {
        vertex.Compute(receivedMessages)
    }
}
```

3.1.4.2 Graph Schema

The first file the user has to modify to program a Go-Pregel algorithm is the graph types file. It has five different GoLang types that should be changed depending on the problem and on how the graph is represented. The first two ones are `VertexIdType` and `EdgeIdType`, which are the types of the vertex and edge IDs, respectively. They can be any type that can be used as a map key in GoLang, but strings are recommended to avoid confusion.

The next two types, `VertexValue` and `EdgeValue`, are the values stored in the vertices and edges. They are the fields that holds any persistent information that the user wants to use in the algorithm. The user can define any fields in these structs, as long as they are serializable by the GoLang json package. This includes the basic types (integer, string, boolean, *etc*), slices or maps of these basic types, or even other structs. The fields defined here will be accessible when programming the algorithm in Section 3.1.4.3. Finally, the last type to be defined by the user is the `PregelMessage` type. This type is the message that is sent from one vertex to another in the superstep. Similarly to `VertexValue` and `EdgeValue`, the `PregelMessage` type can have any fields that are serializable by the GoLang json package.

Although the user is free to use any fields in the `VertexValue` and `EdgeValue` types, these fields will be used to store the initial values of the vertices and edges in the input

graph, if existent. This means that those structs must have, at least, all fields present in the input graph JSON file.

LISTING 3.7 – Go-Pregel types file to be modified by the user.

```
type VertexIdType string // This can be changed
type EdgeIdType string   // This can be changed

type VertexValue struct {
    // Fill in here
}

type EdgeValue struct {
    // Fill in here
}

type PregelMessage struct {
    // Fill in here
}
```

3.1.4.3 Graph algorithm

The second file the user has to modify to program a Go-Pregel algorithm is the methods file. This file has three methods that should be changed depending on the problem and on how the graph is represented. The first one is the `ComputeInSuperStepZero` method, which is called in the first superstep. The second one is the `Compute` method, which is called in the following supersteps. These methods are responsible for interpreting the received messages (if there are any), updating the vertex value if necessary, deciding if the vertex should vote to halt or not, and preparing the messages to be sent to other vertices. The third method is the `CombinePregelMessages`, as explained in Section 3.1.3.1. The Figure 3.8 shows these three aforementioned methods.

LISTING 3.8 – Go-Pregel methods file to be modified by the user.

```
func (vertex *Vertex) ComputeInSuperStepZero() {
    // Fill in here
    /*
        If ComputerInSuperStepZero is the same as Compute,
        you can just call Compute with an empty slice of messages:

        vertex.Compute([]PregelMessage{})
    */
}
```

```
10
11 func (vertex *Vertex) Compute(receivedMessages []PregelMessage) {
12     // Fill in here
13 }
14
15 func CombinePregelMessages(messageList []PregelMessage) []PregelMessage {
16     // Fill in here
17     return messageList // Default behaviour
18 }
```

In order to write the `ComputeInSuperStepZero` and `Compute` methods, the user has access to auxiliary methods that are already implemented in Go-Pregel. These methods are:

- `vertex.GetSuperStepNumber()`: Returns an integer with the current superstep number.
- `vertex.GetValue()`: Returns the current vertex value.
- `vertex.SetValue(VertexValue)`: Sets the vertex value to the value passed as an argument.
- `vertex.GetOutEdges()`: Returns a map of the vertex's out edges.
- `vertex.GetEdgeValue(EdgeIdType)`: Returns the value of the edge with the ID passed as an argument.
- `vertex.SetEdgeValue(EdgeIdType, EdgeValue)`: Sets the edge value to the value passed as an argument.
- `vertex.PrepareMessageToVertex(VertexIdType, PregelMessage)`: This method prepares a message to be sent to another vertex. The message will be sent after the compute method finishes.
- `vertex.VoteToHalt()`: Votes to halt and deactivates the vertex. This vote is automatically withdrawn if the vertex receives a message in the next supersteps.

3.1.5 Go-Pregel Visualizer

The Go-Pregel Visualizer is not a part of the Go-Pregel framework itself, but it is a tool provided to help the user understand the algorithm's output and behaviour. The visualizer is automatically activated by the Go-Pregel scritps after the algorithm finishes, and generates an HTML file with the output graph representation.

The visualizer, differently from the Go-Pregel framework, is written in python. It uses the `json` library to read the output graph file, and the `pyvis` library to generate the HTML file of a graph. The labels of each vertex and edge are customizable through two functions in a separate file. Each function receives the ID of a vertex or edge and its value as a dictionary, and returns a string, defined by the user, to be shown in the image. Figure 3.9 shows the signature of these two functions. An example of the generated images is shown in Section 4.1.2. When opening the file in a browser, the student can inspect the vertices' and edges' IDs and values, drag the vertices' positions with the mouse, zoom in and out of the image, and copy the graph as a PNG file.

LISTING 3.9 – Go-Pregel Visualizer functions to be modified by the user.

```python
def vertex_value_to_display(vertex_id, value_dict) -> str:
    return f"""{vertex_id}""" # This is the default implementation

def edge_value_to_display(edge_id, value_dict) -> str:
    return f"""{edge_id}""" # This is the default implementation
```

3.1.6 Scripts

The project offers three shell scripts that aim to facilitate the use of the framework. The three of them are `start_pregel.sh`, `test_pregel_algorithm.sh`, and `visualize_superstep_state.sh`, each of them with a different purpose and use case. All of them can receive a `-h` or `-help` flag to output a help message, with the available arguments and their descriptions.

3.1.6.1 `start_pregel.sh`

The `start_pregel.sh` script is the main script of the project. It receives the number of workers and the graph file path as arguments, writes the docker compose file, starts the docker containers with the Go-Pregel machines, and after the computation is finished, it calls the visualizer tool to generate the HTML grah file. It also writes the logs from each worker in a separate file, and the master's log is shown in the terminal in real time.

The user can pass three more optional arguments to the script. The first one is the `-debug` flag. If this flag is passed, the Go-Pregel algorithm will be run normally, but the framework will write to storage the subgraphs after each superstep, along with the values in each vertex, their votes to halt, and the messages to be sent. This way, the user can inspect the state of the graph after each superstep and debug the algorithm. If desired, it is possible to visualize these intermediate graph states with the `visualize_superstep_-`

`state.sh` script.

The second optional argument is the `failure_step`. This argument is used to simulate a failure in the computation, and test the framework's fault tolerance. It receives an integer and one of the workers will fail at the superstep passed in this argument.

Finally, the third argument is the `checkpoint_frequency`. This argument is used to set the interval in which the master will create a checkpoint file. If this argument is not passed, Go-Pregel will not create any checkpoint files. Figure 3.10 shows some examples of how to use the script.

LISTING 3.10 – Examples of possible usages of the `start_pregel.sh` script.

```
$ ./start_docker.sh -num_workers=3 -graph_file=sssp/graph1.json

$ ./start_docker.sh -num_workers=3 -graph_file=sssp/graph1.json -failure_step=5
↪  --checkpoint_frequency=2

$ ./start_docker.sh -num_workers=3 -graph_file=sssp/graph1.json -debug
```

3.1.6.2 `test_pregel_algorithm.sh`

The `test_pregel_algorithm.sh` script is used to test the correctness of the user's algorithm. As explained in Section 3.1.1, the repository comes with a directory `graphs/` with other subdirectories with names of graph problems. Each of these directories has json files with example graphs for the problem. The repository also comes with a directory `answers/` with the same structure as the `graphs/` directory, but with the expected output graphs for the problems. The script reads the input graphs from a subdirectory in `graphs/`, runs the user's algorithm for each of them, and compares the output graphs with the ones in the `answers/` directory. Finally, it shows a small log on the terminal, containing how many of the graphs were correctly processed and which of them got incorrect results. The Figure 3.11 shows some examples of valid usages of the script.

This script accepts two mandatory arguments and three optional ones. The two mandatory arguments are `num_workers` (the same as in `start_pregel.sh`) and `algorithm`, which is the subdirectory in `graphs/` where the input graph files are located.

The three optional arguments are `failure_step`, `checkpoint_frequency` and `skip_pregel`. The two first optional arguments have the same meaning as in the `start_pregel.sh` script. The `skip_pregel` argument should only be passed if the user already ran Go-Pregel for all the graphs in the `graphs/` directory, and the output graphs are already in the `src/output_graphs/` directory. If this argument is passed, the script will

skip the Go-Pregel execution and only compare the output graphs with the expected ones.

LISTING 3.11 – Examples of possible usages of the `test_pregel_algorithm.sh` script.

```
$ ./test_pregel_algorithm.sh -num_workers=3 -algorithm=topological_sort

$ ./test_pregel_algorithm.sh -num_workers=3 -algorithm=topological_sort
↪  -failure_step=5 -checkpoint_frequency=2

$ ./test_pregel_algorithm.sh -num_workers=3 -algorithm=topological_sort -verbose

$ ./test_pregel_algorithm.sh -num_workers=3 -algorithm=topological_sort -skip_pregel
```

3.1.6.3 `visualize_superstep_state.sh`

Finally, the `visualize_superstep_state.sh` script is the most simple of the three available scripts. It reads an output graph file or a checkpoint file from the most recent Go-Pregel execution, generates an HTML file with the graph representation, and opens it in the browser. It accepts only two different uses: the user either passes the `superstep` argument to indicate the checkpoint to be visualized, or passes the `output_file` argument to visualize the final output graph from the given file. The Figure 3.12 shows some examples of valid usages of the script.

LISTING 3.12 – Examples of possible usages of the `visualize_superstep_state.sh` script.

```
$ ./visualize_superstep_state.sh -superstep=5

$ ./visualize_superstep_state.sh -output_file=output/graph1.json
```

3.2 Go-Pregel writing guide

Lastly, the project comes with a writing guide that provides a step-by-step method to convert a sequential algorithm to a Go-Pregel algorithm. The guide is in the project's repository, in the file `/pregel_writing_guide.md` This method is based on a set of questions that should be answered by the user, and each answer helps to define a part of the Go-Pregel algorithm. There are three short questions and one more that subdivides into five more questions, in a total of eight.

1. **Is there an initialization step?** This question helps the user to define what happens in `ComputeInSuperStepZero` method.

2. **Are there any inputs to the algorithm, besides the graph itself?** This question helps the user to decide if there are any aditional fields in the `VertexValue` and `EdgeValue` structs, or if there is any global constant to be defined in the algorithm. This is because Go-Pregel doesn't have a way to receive aditional inputs besides the graph.

3. **What is the algorithm output, and what does it represent?** This question helps the user decide on the output graph schema, and what fields should be present in the `VertexValue` and `EdgeValue` structs, regardless of the input graph.

4. **What happens to the algorithm from the point of view of a single vertex?** This question is very generic and is subdivided in five more questions, which help the user to write the `Compute` method.

   (a) **When is a vertex's data accessed or changed by the algorithm?** This question guides to the conditions of message passing and vertex value updating.

   (b) **When does a vertex stop being accessed by the algorithm?** This question guides to the conditions of voting to halt.

   (c) **What happens in each vertex access? What information do we need to process/update it?** This question helps us define what information need to be in the `PregelMessage` struct, and how each vertex interpret these messages.

   (d) **What is the relationship between the order of vertex accesses and the topology of the graph (i.e., the edges)?** This question helps the user to decide the vertices that should receive messages from each vertex in the superstep.

   (e) **Does the order of exploration matter for the algorithm? If so, is there any specific order change that wouldn't change the result?** The answer to this question reveals if a sequential algorithm can be directly translated to a Pregel algorithm, or if there are any changes that should be made.

Although the question 4.e is the last one, it is probably the most important, since if defines if a certain algorithm can be converted into a Go-Pregel algorithm or not. If the graph exploration order determines the result of the algorithm, it isn't possible to directly distribute the algorithm in a Pregel-like framework with considerable gains. However, it doesn't mean that the problem itself can't be solved in Go-Pregel, but the solution will require a different approach. This normally envolves using a different algorithm for the same problem, or creating a Go-Pregel algorithm from scratch. Chapter 4 shows some examples of problem solving with Go-Pregel for different algorithms.

4 Experiments with Go-Pregel

This chapter presents examples of graph problems that will be solved using the Go-Pregel algorithm. For the first algorithm, we will explore the reasoning behind the algorithm conversion, show the final Go-Pregel code in GoLang, and finally, present the end result shown by the visualizer tool. Additionally, we'll see the effects of a machine failure on the algorithm execution, and how the Go-Pregel algorithm handles this situation.

For the other algorithms, we'll limit ourselves to explaining the algorithm writing, and the final Go-Pregel output.

4.1 Single Source Shortest Path

The Single Source Shortest Path (SSSP) problem is a classic graph problem that aims to find the shortest path from a source vertex to all other vertices in a graph. In this problem, each edge has a positive weight (in our work, a positive integer value), and the lenght of a path is the sum of the weights of the edges that make up the path from the source vertex to a target vertex. The most common algorithm to solve this problem is Dijkstra's algorithm.

In Dijkstra's algorithm, we initialize the distance values of each vertex to infinity in a table `dist`, except for the source vertex, which has its distance set to zero. We then instantiate a priority queue with a single pair of values: the source vertex's ID and its distance to itself (zero). We then enter a loop that doesn't end until this priority queue `pq` is empty. In each iteration, we take the next element of `pq` (a vertex u and its distance) and check u's out-neighbors, and if there is an edge (u, v) such that the distance of v is greater than the sum of the distance of u and the weight of (u, v), v's distance is updated so that it is equal to the sum of the other two values, and a new pair (v and its new distance) is added to the priority queue. In the end of this loop, `dist` has the distances of the shortest paths from the source to every other vertex in the graph. Dijkstra's algorithm is shown in Listing 4.1, in a python-like pseudo-code.

LISTING 4.1 – Dijkstra's Algorithm

```python
def dijkstra(graph, source):
    # This is the distance from the source to all other vertices.
    # In the end, this is what we want to return!
    dist = {v: inf for v in graph.vertices}
    dist[source] = 0

    # Priority queue to get the vertex with the smallest distance.
    pq = PriorityQueue()
    pq.push((source, 0))

    # Main loop of the algorithm.
    while not pq.empty():
        u, d = pq.pop()
        for v, w in graph.adj[u]:
            if dist[v] > dist[u] + w:
                dist[v] = dist[u] + w
                pq.push((v, dist[v]))

    # Return the distances.
    return dist
```

However, after going through question 4.e from the guide described in Section 3.2, we notice that the order of the graph exploration must follow the priority queue, making it impossible for us to distribute it directly in Go-Pregel. To solve this problem, we will use the Bellman-Ford algorithm, which is another algorithm that solves the SSSP problem, but with a different approach and a different time complexity.

In Bellman-Ford's algorithm, the distances of each vertex are also initialized in a table `dist` and set to infinity, except for the source vertex, which has its distance set to zero. The algorithm then enters a loop of $N - 1$ iterations, where N is the number of vertices in the graph. Inside this loop, we iterate through all edges in the graph, and check if the distance of the edge's target is greater than the sum of the source's distance and the edge's weight. It positive, the targe's distance is updated to be equal to the sum of the other two values. At the end of the loop, the table `dist` has the desired distances for the SSSP problem. The Bellman-Ford algorithm is shown in Listing 4.2. We now can answer the guide's questions regarding this new algorithm.

LISTING 4.2 – Bellman-Ford Algorithm

```python
def bellman_ford(graph, source):
    # This is the distance from the source to all other vertices.
    # In the end, this is what we want to return!
    dist = {v: inf for v in graph.vertices}
    dist[source] = 0

    # Main loop of the algorithm.
    for _ in range(len(graph.vertices) - 1):
        for u, v, w in graph.edges:
            if dist[v] > dist[u] + w:
                dist[v] = dist[u] + w

    # Return the distances.
    return dist
```

1. **Is there an initialization step?** Yes, we initialize the distances to infinity and the source to 0.

2. **Are there any inputs to the algorithm, besides the graph itself?** Yes, the source vertex.

3. **What is the algorithm output, and what does it represent?** The output is the distance from the source to all other vertices in the graph.

4. **What happens to the algorithm from the point of view of a single vertex?**

   (a) **When is a vertex's data accessed or changed by the algorithm?** A vertex's data is accessed repeatedly in a loop, for every edge that goes out of it.

   (b) **When does a vertex stop being accessed by the algorithm?** A vertex stops being accessed after $N-1$ iterations of the main loop, where N is the number of vertices in the graph.

   (c) **What happens in each vertex access? What information do we need to process/update it?** In each vertex access, which is actually an edge access, the target of the edge is updated if its distance is greater than the source distance plus the edge weight. We need the source distance and the edge weight to update the target's distance.

   (d) **What is the relationship between the order of vertex accesses and the topology of the graph (i.e., the edges)?** None. The algorithm iterates over all edges in the graph, with no particular order.

(e) **Does the order of exploration matter for the algorithm? If so, is there any specific order change that wouldn't change the result?** As long as, at the end of each iteration, each edge has been visited once, the order of exploration doesn't matter.

4.1.1 Writing the Go-Pregel algorithm for SSSP

After going through the questions in the guide as mentioned in Section 3.2, we can confirm that the answer to question 4.e is that the order of the graph exploration doesn't matter in the Bellman-Ford algorithm. This means we can probably translate the algorithm to Go-Pregel without any major changes.

4.1.1.1 Go-Pregel types

After answering question 3, we know that the output of the algorithm must contain the distance from each vertex to the source vertex. Optionally, we can also store in each vertex its predecessor in the shortest path. We also know that the input graph has the weights of the edges as integers. We can then define the types `VertexValue` and `EdgeValue` as shown in Listing 4.3.

Furthermore, through question 4.c, we know that a vertex needs the source distance and the edge weight of its predecessor to update its distance. This information is useful to define the `PregelMessage` type. Since we are storing the ID of the predecessor vertex in the `VertexValue` type, we also need to create a field in the `PregelMessage` type to store the ID of the vertex sending the message. Finally, the `VertexIdType` and `EdgeIdType` types will be left as strings.

LISTING 4.3 – Go-Pregel Value types for the SSSP algorithm

```
1  type VertexIdType string
2  type EdgeIdType string
3
4  type VertexValue struct {
5      Distance    int
6      Predecessor VertexIdType
7  }
8
9  type EdgeValue struct {
10     Weight int
11 }
12
13 type PregelMessage struct {
```

```
14      Sender         VertexIdType
15      SenderDistance int
16      EdgeWeight     int
17  }
```

4.1.1.2 Go-Pregel methods

The first method to implement is the `ComputeInSuperStepZero` method. The answer for question 2 in the guide reveals that there is another input to the algorithm besides the graph: the source vertex. This means that we need to set the source vertex as a global constant. We'll assume that the vertex with ID "0" is always the source vertex.

After going through question 1 and 2, the first part of the implementation of this method is straightforward: for each vertex, we'll set the distance to zero if it is the vertex "0", and to infinity otherwise. We'll represent infinity as the integer -1. We'll also set the predecessor of each vertex to an empty string.

Finally, through question 4.c, we know that, for each edge access, the target vertex is updated depending on the data stored in the edge's origin vertex and edge. This means that, in Go-Pregel, for each vertex, we need to send a message to all its out-neighbors. This way, every edge will be visited once in each superstep.

The second and last method to implement is the `Compute` method. We also write this method's logic based on question 4.c: the vertex is updated based on the information stored in its in-neighbors, which were sent as pregel messages in the previous superstep. Therefore, we iterate over the incoming messages, update the vertex's data accordingly, and send new updated messages to all its out-neighbors.

Finally, question 4.2 guides us to the halting condition. Unfortunately, there isn't a method to retrieve the number of vertices of the graph in Go-Pregel (but this could be done by using an aggregator). However, Bellman-Ford stops after $N-1$ iterations because the algorithm converges after this. In other words, it is possible to use convergence as a halting condition: if a vertex's value doesn't change in a superstep, it should vote to halt. Listing 4.4 shows the implementation of the `ComputeInSuperStepZero` and `Compute` methods. Note that there are edge cases to be considered when involving distances equal to -1.

LISTING 4.4 – Go-Pregel methods for the SSSP algorithm

```
1  func (vertex *Vertex) ComputeInSuperStepZero() {
2      distance := -1
3      if vertex.Id == "0" {
```

```
        distance = 0
    }
    vertex.SetValue(VertexValue{Distance: distance, Predecessor: ""})

    for _, edge := range vertex.GetOutEdges() {
        vertex.PrepareMessageToVertex(
            edge.To,
            PregelMessage{
                Sender:         vertex.Id,
                SenderDistance: distance,
                EdgeWeight:     edge.Value.Weight,
            },
        )
    }
}

func (vertex *Vertex) Compute(receivedMessages []PregelMessage) {
    currentValue := vertex.GetValue()
    currentDistance := currentValue.Distance
    currentPredecessor := currentValue.Predecessor

    hasChanged := false
    for _, message := range receivedMessages {
        if message.SenderDistance == -1 {
            continue
        }
        if currentDistance == -1 || message.SenderDistance+message.EdgeWeight <
        ↪  currentDistance {
            hasChanged = true
            currentDistance = message.SenderDistance + message.EdgeWeight
            currentPredecessor = message.Sender
        }
    }

    if hasChanged {
        vertex.SetValue(VertexValue{Distance: currentDistance, Predecessor:
        ↪  currentPredecessor})
        for _, edge := range vertex.GetOutEdges() {
            vertex.PrepareMessageToVertex(
                edge.To,
                PregelMessage{
                    Sender:         vertex.Id,
                    SenderDistance: currentDistance,
                    EdgeWeight:     edge.Value.Weight,
                },
            )
```

```
48          }
49      } else {
50          vertex.VoteToHalt()
51      }
52  }
```

4.1.1.3 Visualizer tool functions

The visualizer tool customizable functions don't affect the algorithm's execution whatsoever. However, they are useful to visualize the algorithm's output in a friendly way. In this problem, we are interested in the `Distance` and `Predecessor` fields of the `VertexValue` type. The customization of the visualizer functions is simple, and shown in Listing 4.5.

LISTING 4.5 – Visualizer tool functions for the SSSP algorithm

```
1  def vertex_value_to_display(vertex_id, value_dict) -> str:
2      predecessor = value_dict["Predecessor"]
3      if predecessor == "":
4          predecessor = "None"
5      return f"""{vertex_id}\nDistance: {value_dict["Distance"]}\nPredecessor:
       ↪  {predecessor}"""
6
7
8  def edge_value_to_display(edge_id, value_dict) -> str:
9      return f"""{edge_id}"""
```

4.1.2 Running the SSSP algorithm

After implementing the Go-Pregel algorithm for the SSSP problem, we can use the provided scripts to run it and visualize the results. Listing 4.6 shows the most important terminal logs of the Go-Pregel execution with 3 workers in a small graph. Note that four supersteps were executed to end the algorithm. The visualizer tool's output is shown in Figure 4.1. In order to maintain readability of Figure 4.1, the input graph used has all its edges with weight 1, and the weights are ommited from the image.

LISTING 4.6 – SSSP algorithm execution logs

```
$ ./start_pregel.sh -num_workers=3 -graph_file=sssp/graph1.json
(...)
  Container pregel-master    Started                                          0.3s
  Container pregel-worker-2  Started                                          0.2s
  Container pregel-worker-3  Started                                          0.2s
  Container pregel-worker-1  Started                                          0.2s
Starting Pregel with 3 workers on file sssp/graph1.json
2024/11/16 19:01:18 Registering worker '0' with hostname 'pregel-worker-2:50002'
2024/11/16 19:01:18 Registering worker '1' with hostname 'pregel-worker-3:50003'
2024/11/16 19:01:18 Registering worker '2' with hostname 'pregel-worker-1:50001'
2024/11/16 19:01:27 Checking workers
2024/11/16 19:01:27 Partitioning graph
2024/11/16 19:01:27 Partitioning graph between 3 workers
2024/11/16 19:01:27 Ordering workers to execute superstep
2024/11/16 19:01:27 Ordering workers to execute superstep
2024/11/16 19:01:27 Ordering workers to execute superstep
2024/11/16 19:01:27 Ordering workers to execute superstep
2024/11/16 19:01:27 Ordering workers to write subgraphs
2024/11/16 19:01:27 Reducing subgraphs and writing to file
2024/11/16 19:01:27 ./output_graphs/SubGraph-0.json
2024/11/16 19:01:27 ./output_graphs/SubGraph-1.json
2024/11/16 19:01:27 ./output_graphs/SubGraph-2.json
2024/11/16 19:01:27 Ordering workers to finish operations
2024/11/16 19:01:27 Pregel finished
Stopping Pregel containers
Stopping all containers based on the image pregel
graph.html
Opening graph image in browser...
```

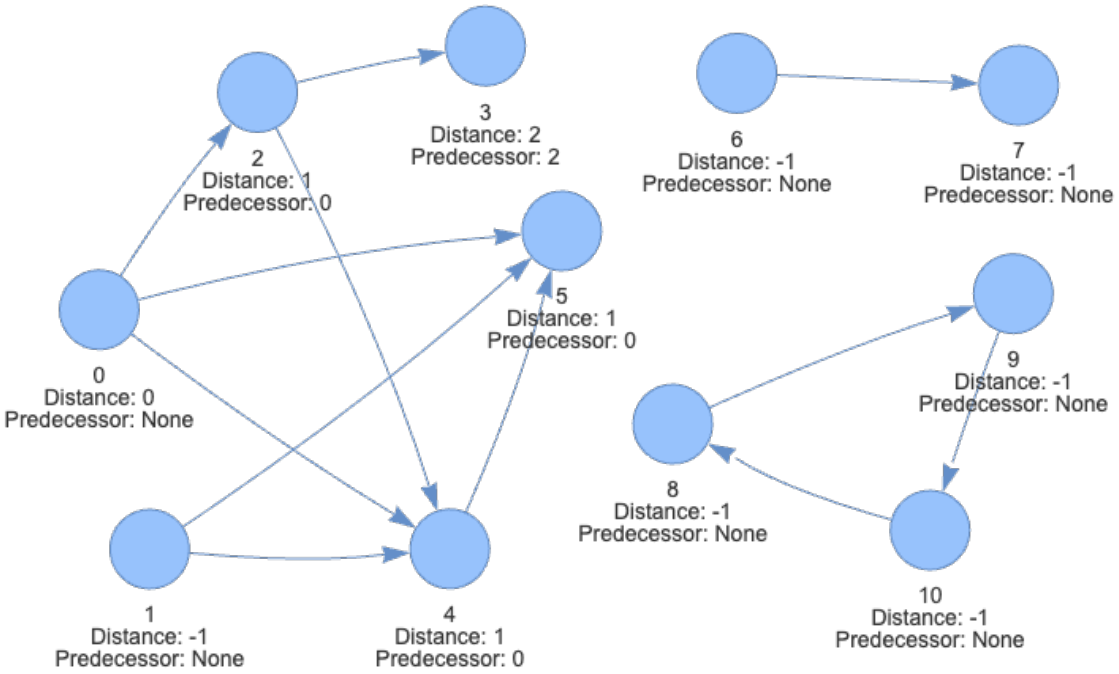


FIGURE 4.1 – SSSP output generated by the visualizer tool

As expected, the distance from the source vertex to itself is zero, and the distances to unreachable vertices are still -1 (infinity). The image also shows the predecessor of each vertex in the shortest path from the source vertex. The optimal path from vertex 0 to vertex 3, for example, can be derived by following the predecessor field, from vertex 3 to vertex 0. The predecessor of vertex 3 is vertex 2, and the predecessor of vertex 2 is vertex 0. Therefore, the shortest path from vertex 0 to vertex 3 is going from vertex 0 to vertex 2, and then from vertex 2 to vertex 3, with a total distance of 2.

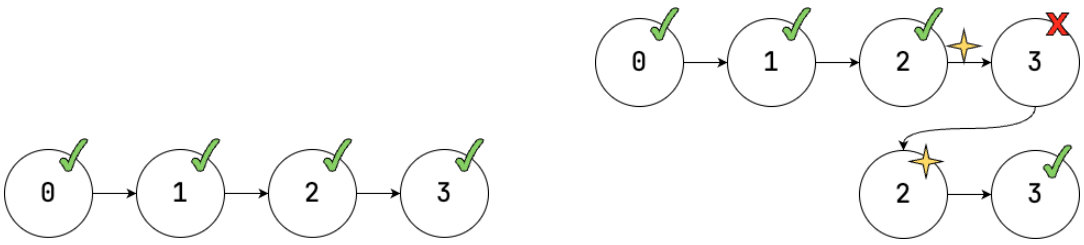
4.1.3 Testing fault tolerance

We can also test the fault tolerance of the framework by using the `-failure_step` and `-checkpoint_frequency` flags in the `start_pregel.sh` script. We'll set the failure step to 3 and the checkoint frequency to 2. This means that one of the workers will fail at superstep 3, and a checkpoint will be saved every two supersteps. Listing 4.7 shows the most important terminal logs for the same execution as in Listing 4.6, but with a worker failure at superstep 3, and the checkpoint frequency set to 2.

LISTING 4.7 – SSSP algorithm execution with failure

```
$ ./start_pregel.sh -num_workers=3 -graph_file=sssp/graph1.json -failure_step=3
↪  -checkpoint_frequency=2
(...)
2024/11/09 17:19:29 Partitioning graph between 3 workers
2024/11/09 17:19:29 Ordering workers to execute superstep
2024/11/09 17:19:29 Ordering workers to execute superstep
2024/11/09 17:19:29 Ordering workers to execute superstep
2024/11/09 17:19:29 Ordering workers to write subgraphs
2024/11/09 17:19:29 Ordering workers to execute superstep
2024/11/09 17:19:29 Failed to order superstep to worker. Error: (...)
2024/11/09 17:19:29 Checking workers
2024/11/09 17:19:29 Failed to check worker with ID 0. Error: (...)
(...)
2024/11/09 17:19:29 Next superstep:  3
2024/11/09 17:19:29 Partitioning graph between 2 workers
2024/11/09 17:19:29 Ordering workers to execute superstep
2024/11/09 17:19:29 Ordering workers to write subgraphs
(...)
2024/11/09 17:19:29 Pregel finished
```

Note that, as expected, the supersteps 0, 1, and 2 were executed normally, and after superstep 2, the master machine ordered the workers to write a checkpoint ("write subgraphs"). After that, the workers attempted to execute superstep 3, but one of them failed. The master machine then noticed the failure, checked the remaining workers, repartitioned the graph from the last checkpoint, and ordered the remaining workers to execute superstep 3 again. The algorithm then continued normally, and the final output was the same as in Figure 4.1. Figure 4.2 shows the progression of the supersteps without failure and with failure.



(a) No failure.

(b) Failure in superstep 3. The yellow star is a checkpoint made after superstep 2.

FIGURE 4.2 – Superstep progressions with and without failure.

4.2 Connected Components

In Section 4.1, we showed the conversion of the Bellman-Ford algorithm to Go-Pregel by using the guide from Section 3.2. However, there are problems in which the best and easiest alternative is to create an algorithm from scratch, which relies on the understanding of the problem and on the user's creativity. We'll use this approach to solve problems from Section 4.2 onwards.

In this problem, we desire to find the weakly connected components (WCC) of a graph, and identify each vertex with a unique ID representing its WCC. The first thing to do when solving this problem is how to define the WCC ID. For simplicity, we can, for example, simply use one of the vertex IDs in the WCC, specifically the lexicographically smallest one, considering that the vertex IDs are strings.

We now desire to define the smallest vertex ID in each WCC. To do so, each vertex can, in each superstep, send its "opinion" on what the smallest vertex ID to its neighbors. At first, the vertices will claim to have the smallest ID, but, if they receive a message from a neighbor with a smaller ID, they will update their opinion. This information will be then propagated until all vertexes in the WCC agree on the smallest ID. However, this solution raises a problem: each vertex has to send messages to its neighbors, and this includes out-neighbors and in-neighbors. The out-neighbors can be easily accessed through the vertex's list of out-edges. However, the in-neighbors are not provided in Go-Pregel.

In order to solve this problem, we can make a small change in the algorithm: in the superstep zero, each vertex will send its ID to all its out-neighbors as their opinion. In superstep one, each vertex will receive the IDs of its in-neighbors and, besides updating its opinion on the smallest ID, it will also store each received ID in a list. This list contains the IDs of all its in-neighbors. The rest of the algorithm is the same as before, since all vertices now have the IDs of their in-neighbors stored. If a vertex's opinion doesn't change in a superstep, it should vote to halt and should not send any messages.

As a final correction to the algorithm, a vertex should only vote to halt from superstep two onwards. Halting in superstep one leads to incorrect results in an edge case: if there are two vertices 1 and 2, and an edge from 2 to 1, vertex 1 will not change its opinion on the WCC ID, so it will halt and not send any messages. Vertex 2 will then not receive any messages, and will also halt while still having the wrong ID. This only happened because vertex 1 didn't have the ID of its in-neighbor stored in superstep zero, which is not true in superstep one.

The fields necessary in the Go-Pregel algorithm are shown in Listing A.1 in the Appendix, and the code for the `ComputeInSuperStepZero` and `Compute` methods are shown in Listing A.2. The final output of the algorithm is shown in Figure 4.3.

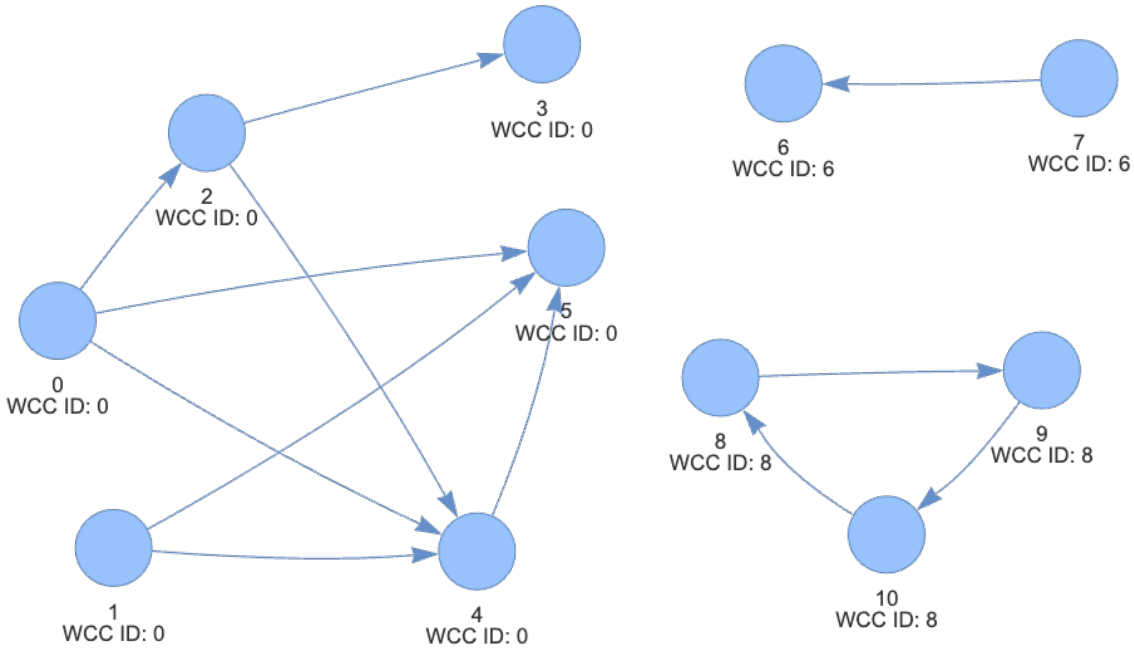


FIGURE 4.3 – Connected Components output generated by the visualizer tool

4.3 Topological Sort

In this problem, we desire to sort the vertices of a directed acyclic graph (DAG) in a way that, for each edge (u, v), u comes before v in the sorted list. In the Go-Pregel context, we will assign a number to each vertex, representing its order in the topological sort, with the difference that we will allow ties to happen. In other words, if there are two vertices u and v that have no in-neighbors, both u and v will receive the number zero in the topological sort.

This problem is simpler than the previous one. We can assume, from the beginning, that every vertex has the same topological order equal to 0, and fix the order over the supersteps. In the end of each superstep, each vertex will send a message to its out-neighbors with its current topological order. If a vertex receives a message with a number greater than or equal to its current order, it will update its order (it will be the order in the message plus one) and send new messages to its out-neighbors. Finally, if a vertex's order doesn't change in a superstep, it votes to halt and doesn't send any messages. Figure 4.4 shows the final output of the algorithm, and Listings A.3 and A.4 show the implementation of the Go-Pregel types and methods for this algorithm.

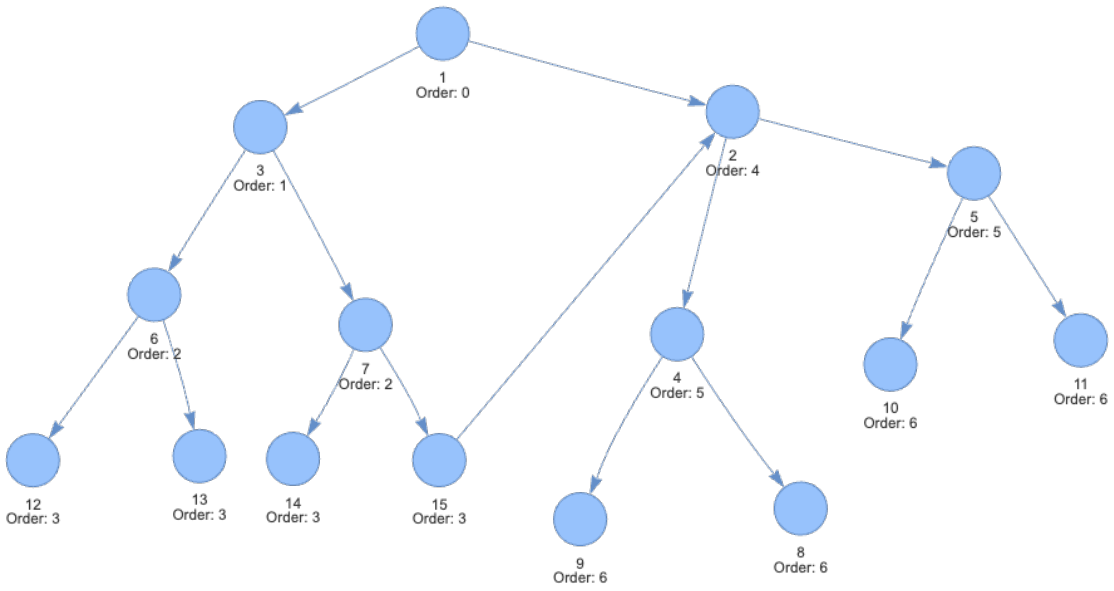


FIGURE 4.4 – Topological Sort output generated by the visualizer tool

4.4 Graph Bipartition

The bipartition problem, (a particular case of the coloring problem, but with two colors), consists on dividing the graph's vertices into two disjoint sets, such that no two vertices in the same set are adjacent. If such division is possible, the graph is said to be bipartite. In this problem, we will assign a partition number (or color) to each vertex, either 0 or 1, representing the two sets, and, if a weakly connected component is not bipartite, its vertices will receive a third partition number, -1.

At first glance, this problem seems to be similar to the topological order algorithm, in the sense that we could start by assigning the same color to all vertices, and then propagate the colors through messaging. However, a problem arises when updating the colors in a superstep: if two adjacent vertices send messages to each other indicating their color, they would both update their color to the same value, and the same thing would happen in the following superstep, resulting in an infinite number of supersteps. That is why, for this problem, besides the color, one more field is necessary to be sent in the Go-Pregel message: a `SourceOfTruth` field. For each bipartite weakly connected component, we'll fix the color of the vertex with the smallest ID to 0.

Similarly to the Weakly Connected Components problem, we need to treat the input graph as if it were its underlying undirected graph, so we will also store a `neighbors` set

in the vertex value. In superstep zero, we'll set every vertex's color to 0, and the source of truth as its ID. We'll then send a message to its out-neighbors with its color in a field `SenderPartition`, as well as the vertex's ID as the source of truth. We'll also store each out-neighbor's ID in the neighbors list.

In each superstep from superstep one onwards, we'll create a `hasChanged` boolean variable set to false, and then iterate through the messages the vertex received. If it is the superstep one (remember that the first superstep is superstep zero), we'll store the message's source of truth in the neighbors list, because in this superstep, the source of truth is also the message sender, and it is an in-neighbor. The neighbors list now has every neighbor's ID stored.

Still in the iteration through the received messages, but outside the superstep one condition, we take different actions depending on the message's content and the current vertex value, as in the following list. If an item's condition is met, we execute its action and proceed to the next received message.

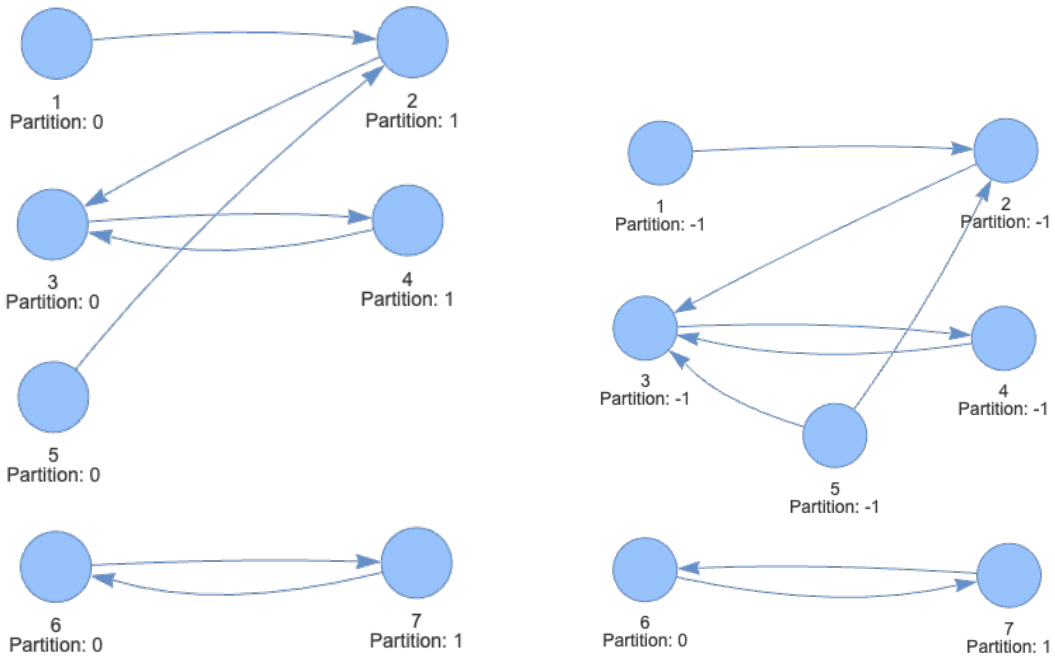
1. If the vertex's current color is -1: set `hasChanged` to true if and only if the message sender's color is different than -1.
2. If the sender's color is -1 (which means that the connected component is not bipartite): we update the vertex's color to -1, the source of truth to the message's source of truth, and set `hasChanged` to true.
3. If the message's source of truth is smaller than the vertex's source of truth: we update it to the minimum of the two, and set the vertex's color to the opposite of the message sender's color. We set `hasChanged` to true.
4. If the vertex's color is equal to the message sender's color and the vertex's source of truth is the same as the message's source of truth: we set `hasChanged` to true, and the vertex's color to -1.

After iterating through all received messages, if `hasChanged` is true or if the algorithm is in superstep one, we send messages to all the vertex's neighbors with the updated color and source of truth. Otherwise, the vertex votes to halt and doesn't send any messages.

Note that the last item in the previous list is what determines if a connected component is bipartite or not. If a vertex receives a message from a neighbor with the same color and the same source of truth, we know that a vertex received different colors from the same vertex in different supersteps, through different paths, which means that the connected component is not bipartite.

We show, in Figure 4.5, the output of this Go-Pregel algorithm for two similar graphs. The first one is a bipartite graph with seven vertices. The second one is almost identical

to the first one, but with an aditional edge that makes one of the connected components non bipartite.



(a) Output for a bipartite graph.

(b) Output for a non-bipartite graph.

FIGURE 4.5 – Bipartite graph algorithm outputs generated by the visualizer tool.

Listings A.5 and A.6 show the implementation of the Go-Pregel types and methods for the bipartite graph algorithm.

5 Educational Practice with Go-Pregel

After developing the Go-Pregel framework, we conducted a practical activity with students from the distributed processing course from the Aeronautics Institute of Technology (ITA) in São José dos Campos, Brazil. The activity aimed to introduce the students to the importance of distributed processing in graphs and to Pregel-like frameworks. This activity was divided into three parts, which will be explained in the following sections.

5.1 Pre-Lab Activity

Before the practical activity, we gave a short pre-lab guide for the students to read and follow. The guide is available at the project's repository under the `/lab-for-students/prelab/` directory. The pre-lab was a short preparation for the practical activity, and was basically the installation steps of the Go-Pregel project. There were four mains steps:

1. Clone the Go-Pregel repository from GitHub;
2. Install docker and the necessary python libraries;
3. Make small changes in the script depending on the operating system (two of the commands were different for Windows, Linux and MacOS);
4. Run Go-Pregel on a given graph.

The last step doesn't require any knowledge on how Go-Pregel works, as the students were only required to run the Go-Pregel script on a given graph, without the need to modify the files that determine the graph algorithm. Since those files were remained unchanged, the vertices were set to vote to halt immediately, and a graph HTML file would be opened on the student's browser. This image would only be opened if all installation steps were correctly followed, so this step is a simple way to check if the pre-lab was successful.

5.2 Activity with Go-Pregel

On the day of the activity, the students were given a 40-minute lecture on distributed graph processing frameworks, with focus on Go-Pregel. The lecture also contained an explanation of the guide from Section 3.2, as well as the example from Section 4.1. After the lecture, the students were divided into pairs, and each pair could choose two problems among three to solve using Go-Pregel. The problems were the ones explained in Sections 4.2, 4.3 and 4.4. The complete lab guide, containing the activities to be done, is available at the project's repository under the `/lab-for-students/lab/` directory.

The students were then given one week to solve the problems. They were required to return the code with the solution and an explanation of how they solved the problem, after testing their solution on the given input graphs.

5.3 Feedback form

After finishing the activity, we asked the students to fill a feedback form about the lecture and activity. The form contained 13 questions about the subject's relevance, the interest in the subject, instructions clarity, learning experience and general satisfaction. All questions were in form of a sentence, and the students could assign integers from 1 to 5 to each sentence, meaning "disagree", "partially disagree", "neutral", "partially agree" and "agree", respectively. However, when analysing the responses, we grouped the answers in only three categories: 1 or 2 ("disagree"), 3 ("neutral"), and 4 or 5 ("agree"). The form also had a space for the students to write suggestions and comments about the activity. The sentences given to the students were:

1. I found the subject relevant to the current technological scenario;
2. I will use the knowledge acquired about Pregel after graduating from college;
3. The activity was interesting and engaging;
4. The activity maintained my attention during the whole time;
5. I found myself more motivated to learn about graph processing after the activity;
6. The examples given in the lecture were interesting and relevant;
7. I liked the activity;
8. I am now confident on my understanding about graph processing because of the activity;

9. The instructions of the activity were clear;
10. The pace of the activity was adequate for my level of knowledge on the subject;
11. The activity helped me to understand how to implement graph algorithms in a distributed environment;
12. The activity was well balanced between theory and practice;
13. I would recommend this activity to other students that want to learn about distributed graph processing;

The graph from Figure 5.1 shows the answers from the feedback form grouped by question. The horizontal axis represents the questions, and the vertical axis represents the number of answers of each category (agree, neutral, disagree). The questions are represented by the numbers from 1 to 13, following the order from the list of questions above, and the categories are represented by the colors green (agree), yellow (neutral) and red (disagree).

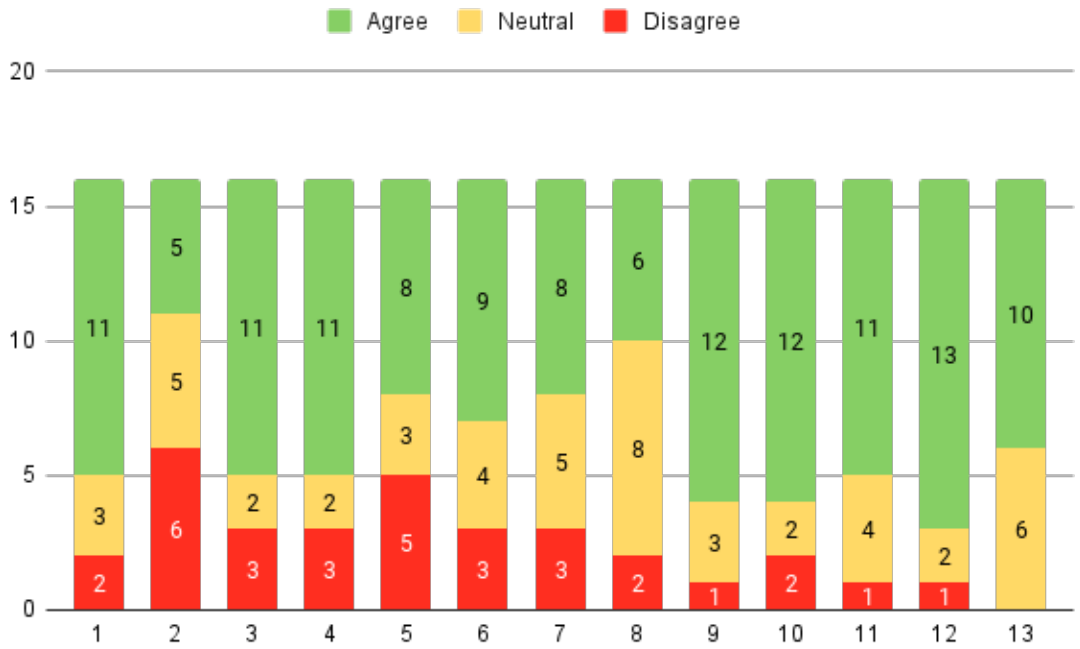


FIGURE 5.1 – Answers from the feedback form

The two first questions revealed an interesting dichotomy. While most students found the subject relevant to the current technological scenario, most of them stood neutral or disagreed with the statement that they would use the knowledge acquired about Pregel after graduating from college. This is understandable since distributed processing in graphs is a very specific subject, although being highly relevant to the industry.

A similar situation happened with the questions from 3 to 6, which were about the interest in the activity. Most students agreed that the activity was interesting and engaging, and most had their attention retained during the whole time. However, not so many of them found themselves to be more motivated to learn about graph processing after the activity. Again, this can be explained by the specificity of the subject: although the subject itself was interesting, the probability of them working with distributed graph processing in the future is not high.

Even then, the students claimed to like the activity and to be confident in their new knowledge about graph processing. This means the overall objective of the activity was achieved, as the students were able to learn about distributed graph processing and to implement some algorithms using Go-Pregel. All questions about learning experience and instructions clarity had positive feedback (questions 9 to 13).

Finally, there was an optional space for the students to write suggestions and comments about the activity and the project, but we received only two comments. Student A stated that the he wished to have a more detailed explanation of the situations in which it is advantageous to use a distributed graph processing algorithm instead of a centralized one. The other comment, from student B, was about the Go-Pregel project itself: the student was highly impressed by the simplicity of the framework, and liked that one could implement a distributed graph algorithm by changing only two functions. However, the same student reported that people who missed the lecture had difficulty with the activity, as the guide was not enough to explain how to use the framework. This only reinforces the importance of understanding how distributed processing frameworks work before using them.

6 Conclusions

In this chapter, we present the conclusions of this work, as well as the future work that can be done to improve the Go-Pregel framework.

6.1 Discussions

After finishing the development of the framework and conducting the practical activity with the students, we were able to draw some conclusions about both the project itself and the proposed activity.

The Go-Pregel framework itself proved to work properly, and it was simple enough to use, as the students were able to implement two classic graph algorithms in Go-Pregel, with only a short lecture as preparation. However, there were some limitations in the framework that limit its generality. For example, the framework allows the implementation of only static graphs, whereas some algorithms involve changes in the graph topology during the computation. The Minimum Spanning Tree problem, for example, requires the removal of edges from the graph, and other algorithms such as k-core finding problem require the removal of vertices.

Removing vertices and edges from the graph sounds simple, but it does bring some challenges to the framework's implementation, most of them related to possible ambiguities caused by such scenarios. For example, if two vertices issue requests to create a vertex in the same superstep, we must have a way to decide which of the two requests will be accepted, if any. In a similar way, if there are two different requests to create an edge between two vertices, we must decide if only one edge will be created (and one of the requests will be considered redundant), or if the algorithm should create two different edges between the same vertices. This decision is not trivial, and some algorithms may require different behaviors.

The original Pregel framework provides different handlers for each of those situations with a standard behaviour, but the user can override each one of them if necessary. One big challenge to implement dynamic graph processing in Go-Pregel is how to provide such

flexibility to the user without making the framework too complex. Full flexibility and generality comes with the cost of programming complexity, which can be counterproductive if the goal of the project is to teach the basics of distributed graph processing.

Aditionally, there were two interesting features that could be added and explored in an activity: combiners and aggregators. Although the Go-Pregel framework comes with a simple combiner function that can be modified by the user if desired, we didn't explore this feature in the activity. The current Go-Pregel doesn't measure the time taken by the computation step, or the time in message exchange, so the effect of combiners in the performance of the algorithm cannot be measured by the user. Measuring such effects would be an important feature to motivate the students to use combiners in their algorithms.

On the other hand, aggregators were not implemented in the Go-Pregel framework, but they could be an interesting feature to help the user solve simple or even complex problems. The main challenge in implementing aggregators is to design an architecture that is capable of receiving an indefinite number of different aggregators, set by the user, while maintaining the simplicity of the project. It is also important that the Go-Pregel writing guide contains instructions on how (and when) to use both combiners and aggregators.

Adding these features to the Go-Pregel framework (dynamic graphs and aggregators) would allow us to expand the range of classic problems that can be solved using our software, creating a sense of freedom to the students submitted to the activity, which could improve their learning experience.

When it comes to fault tolerance, the project provides a checkpointing solution similar to Google's Pregel system. However, Go-Pregel has a clear limitation: the graph is not redistributed if a worker is reactivated after a failure. This means that the number of workers is always decreasing in case of failure, and a new worker can never join the computation after the algorithm starts. On the other hand, repartitioning the graph everytime a new worker joins the algorithm would cause an unnecessary overhead. If worker reconnection is implemented in the future, is is probably advantageous to create conditions to decide when a reconnection is allowed. Some possible conditions are, for example, reconnecting when there is another failure or after a new checkpoint is created.

Another point to be discussed about the project is the Pregel algorithm writing guide provided to the students. The guide was written under the assumption that the best way to write Pregel algorithms was to directly translate an already existing centralized algorithm through a series of questions and answers. Although this worked for some algorithms, the activity showed that the students found it easier to write the algorithm from scratch than following the guide, because doing so normally required the student to search for an specific algorithm that could be directly converted to Go-Pregel. This showed that,

rather than translating an existing algorithm, the guide should focus on teaching how to think and create a Pregel algorithm based on the problem's requirements.

Finally, the execution of the activity with the students showed that the Go-Pregel instalation procedure was not simple enough and caused problems to some of them. Most of the problems were related to the students' using a different operating system. Although the installation guide listed the necessary changes for each OS, having to modify lines of scripts manually led to forgetting some of these changes, which caused unexpected errors when running Go-Pregel. A possible solution is to update these scripts to automatically detect the current OS and make the correct decisions.

6.2 Final Remarks

Despite all the limitations of the current state of the Go-Pregel framework, the project was able to achieve its main goal: to provide a simple and easy-to-use framework for teaching distributed graph processing. Go-Pregel not only allowed the students to implement classic graph algorithms in it, but it also includes a fault tolerance solution, which is an important feature for a distributed system. Furthermore, the project also provides scripts with different options and use cases, along with an interactive visualization tool that takes any output graph and displays a customizable representation of it in a browser.

6.3 Future Work

Go-Pregel is a project that, in a way, is still in its early stages. There are many opportunities of improvement and expansion of the project, most of them related to the limitations described in Section 6.1. The main points that can be improved, in order of descending priority, are:

1. Rethink and update the Go-Pregel algorithm writing guide: since the current guide showed limitations in terms of universality, it should be rewritten to optimize the students' learning experience.

2. Implement aggregators: aggregators can be a powerful tool to solve complex problems, and should be an interesting feature to add to the project.

3. Implement support for dynamic graph processing: this ultimately comes down to allowing the user to remove and create vertices and edges during the execution of the algorithm.

4. Machine reconnection in fault tolerance: the framework should allow a new machine to connect to the computation after the beginning of the algorithm, under certain conditions.

5. Finally, the number of example input graphs for each algorithm, as well as the number of algorithms, should increase to provide a more complete experience of the Go-Pregel usage. This should happen naturally when implementing features like dynamic graphs and aggregators.

Bibliography

APACHE Giraph. August 2020. Available at: https://giraph.apache.org/. Accessed on: June 13, 2024.

CHEATHAM, T.; FAHMY, A.; STEFANESCU, D. C.; VALIANT, L. G. Bulk synchronous parallel computing - a paradigm for transportable software. **Harvard Computer Science Group Technical Report**, n. TR-36-94, 1994. Available at: https://nrs.harvard.edu/urn-3:HUL.InstRepos:26506457.

CHING, A.; LOGOTHETIS, D.; EDUNOV, S.; MUTHUKRISHNAN, S.; KABIJO, M. One trillion edges: Graph processing at facebook-scale. **Proceedings of the VLDB Endowment**, Volume 8, August 2015. Available at: https://dl.acm.org/doi/10.14778/2824032.2824077.

FORTUNATO, S.; HRIC, D. Community detection in networks: A user guide. **Physics Reports**, v. 659, 07 2016.

GANDOUR, G. **Go Pregel**. 2024. Available at: https://github.com/GaGandour/Go-Pregel.

GONZALEZ, J. E.; LOW, Y.; GU, H.; BICKSON, D.; GUESTRIN, C. Powergraph: Distributed graph-parallel computation on natural graphs. **USENIX Symposium on Operating Systems Design and Implementation**, October 2012. Available at: https://www.usenix.org/system/files/conference/osdi12/osdi12-final-167.pdf. Accessed on: June 14, 2024.

GONZALEZ, J. E.; XIN, R. S.; DAVE, A.; CRANKSHAW, D.; FRANKLIN, M. J.; STOICA, I. Graphx: Graph processing in a distributed dataflow framework. **USENIX Symposium on Operating Systems Design and Implementation**, October 2014. Available at: https://www.usenix.org/conference/osdi14/technical-sessions/presentation/gonzalez. Accessed on: June 14, 2024.

HEIDARI, S.; SIMMHAN, Y. L.; CALHEIROS, R. N.; BUYYA, R. Scalable graph processing frameworks. **ACM Computing Surveys (CSUR)**, v. 51, p. 1 – 53, 2018. Available at: https://api.semanticscholar.org/CorpusID:3925107.

KOLDEHOFE, B.; PAPATRIANTAFILOU, M.; TSIGAS, P. Distributed algorithms visualisation for educational purposes. **SIGCSE Bull.**, Association for Computing Machinery, New York, NY, USA, v. 31, n. 3, p. 103–106, jun 1999. ISSN 0097-8418. Available at: https://doi.org/10.1145/384267.305884.

LI, M.; ZHANG, J.; WAN, J.; REN, Y.; ZHOU, L.; WU, B.; YANG, R.; WANG, J. Distributed machine learning load balancing strategy in cloud computing services. **Wireless Networks**, 2020.

LI, X.; MENG, K.; QIN, L.; LAI, L.; YU, W.; QIAN, Z.; LIN, X.; ZHOU, J. Flash: A framework for programming distributed graph processing algorithms. **39th IEEE International Conference on Data Engineering**, August 2022. Available at: https://lai.me/publication/xue-2023-icde/.

MALEWICZ, G.; AUSTERN, M. H.; BIK, A. J.; DEHNERT, J. C.; HORN, I.; LEISER, N.; CZAJKOWSKI, G. Pregel: a system for large-scale graph processing. *In*: **Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data**. **Proceedings** [...]. New York, NY, USA: Association for Computing Machinery, 2010. (SIGMOD '10), p. 135–146. ISBN 9781450300322. Available at: https://doi.org/10.1145/1807167.1807184.

MICHAILIDIS, G.; LEEUW, J. de. Data visualization through graph drawing. **Computational Statistics**, v. 16, n. 3, p. 435–450, 2001. Available at: https://doi.org/10.1007/s001800100077.

PAGE, L.; BRIN, S.; MOTWANI, R.; WINOGRAD, T. **The PageRank Citation Ranking: Bringing Order to the Web**. 1998. Available at: https://api.semanticscholar.org/CorpusID:1508503.

PENG, C.; XIA, F.; NASERIPARSA, M.; OSBORNE, F. Knowledge graphs: Opportunities and challenges. **Artificial Intelligence Review**, 2023.

SHIRINIVAS, S. G.; VETRIVEL, S.; ELANGO, N. M. Applications of graph theory in computer science an overview. **International Journal of Engineering Science and Technology**, v. 2, n. 9, 2010.

WANG, X.; ZHANG, Y.; QIN, L.; WEN, D.; CHANG, L.; LIN, X. Graph3s: A simple, speedy and scalable distributed graph processing system. **PVLDB**, March 2020. Available at: https://arxiv.org/abs/2003.00680.

WEST, D. B. **Introduction to Graph Theory**. Second edition. 482 F.I.E. Patparganj, Delhi 110 092, India: Pearson Education, Inc., 2001.

WILSON, R. J. **Introduction to Graph Theory**. Fourth edition. Edinburgh Gate, Harlow, Essex CM20 2JE, England: Addison Wesley Longman Limited, 1996.

YAN, D.; CHENG, J.; XING, K.; LU, Y.; NG, W.; BU, Y. Pregel algorithms for graph connectivity problems with performance guarantees. **PVLDB**, v. 7, n. 14, September 2014. Available at: https://www.vldb.org/pvldb/vol7/p1821-yan.pdf.

ZHU, X.; CHEN, W.; ZHENG, W.; MA, X. Gemini: A computation-centric distributed graph processing system. **USENIX Symposium on Operating Systems Design and Implementation**, November 2016. Available at: https://www.usenix.org/system/files/conference/osdi16/osdi16-zhu.pdf. Accessed on: June 14, 2024.

Appendix A - Go-Pregel algorithm codes for the problems in Chapter 4

This appendix contains the Go-Lang implementations of the Go-Pregel algorithms explained in Chapter 4.

A.1 Codes for the Connected Components problem

LISTING A.1 – Go-Pregel types for the Connected Components algorithm

```
type VertexIdType string
type EdgeIdType string

type VertexValue struct {
    ComponentID VertexIdType
    Neighbors   map[VertexIdType]bool
}

type EdgeValue struct {
    // Empty
}

type PregelMessage struct {
    ComponentID VertexIdType
}
```

LISTING A.2 – Go-Pregel methods for the Connected Components algorithm

```
func (vertex *Vertex) ComputeInSuperStepZero() {
    neighbors := make(map[VertexIdType]bool)

    for _, edge := range vertex.GetOutEdges() {
        neighbors[edge.To] = true
```

```
6           vertex.PrepareMessageToVertex(
7               edge.To,
8               PregelMessage{
9                   ComponentID: vertex.Id,
10              },
11          )
12      }
13
14      vertex.SetValue(
15          VertexValue{
16              ComponentID: vertex.Id,
17              Neighbors:   neighbors,
18          },
19      )
20  }
21
22  func (vertex *Vertex) Compute(receivedMessages []PregelMessage) {
23      vertexValue := vertex.GetValue()
24      neighbors := vertexValue.Neighbors
25      componentID := vertexValue.ComponentID
26      changed := false
27
28      for _, message := range receivedMessages {
29          if vertex.GetSuperStepNumber() == 1 {
30              neighbors[message.ComponentID] = true
31          }
32          messageIntComponentID, _ := strconv.Atoi(string(message.ComponentID))
33          componentIDInt, _ := strconv.Atoi(string(componentID))
34          if messageIntComponentID < componentIDInt {
35              componentID = message.ComponentID
36              changed = true
37          }
38      }
39
40      if changed || vertex.GetSuperStepNumber() == 1 {
41          vertex.SetValue(
42              VertexValue{
43                  ComponentID: componentID,
44                  Neighbors:   neighbors,
45              },
46          )
47          for neighbor := range neighbors {
48              vertex.PrepareMessageToVertex(
49                  neighbor,
50                  PregelMessage{
51                      ComponentID: componentID,
```

```
                },
            )
        }
    } else {
        vertex.VoteToHalt()
    }
}
```

A.2 Codes for the Topological Sort problem

LISTING A.3 – Go-Pregel types for the Topological Sort algorithm

```
type VertexIdType string
type EdgeIdType string

type VertexValue struct {
    TopoOrder int
}

type EdgeValue struct {
}

type PregelMessage struct {
    SenderOrder int
}
```

LISTING A.4 – Go-Pregel methods for the Topological Sort algorithm

```
func (vertex *Vertex) ComputeInSuperStepZero() {
    vertex.SetValue(
        VertexValue{
            TopoOrder: 0,
        },
    )
    for _, edge := range vertex.GetOutEdges() {
        vertex.PrepareMessageToVertex(
            edge.To,
            PregelMessage{
                SenderOrder: 0,
            },
        )
    }
```

```
}

func (vertex *Vertex) Compute(receivedMessages []PregelMessage) {
    currentOrder := vertex.GetValue().TopoOrder
    hasChanged := false
    for _, message := range receivedMessages {
        if message.SenderOrder >= currentOrder {
            hasChanged = true
            currentOrder = message.SenderOrder + 1
        }
    }

    if hasChanged {
        vertex.SetValue(
            VertexValue{
                TopoOrder: currentOrder,
            },
        )
        for _, edge := range vertex.GetOutEdges() {
            vertex.PrepareMessageToVertex(
                edge.To,
                PregelMessage{
                    SenderOrder: currentOrder,
                },
            )
        }
    } else {
        vertex.VoteToHalt()
    }
}
```

A.3 Codes for the Graph Bipartition problem

LISTING A.5 – Go-Pregel types for the Graph Bipartition algorithm

```
type VertexIdType string
type EdgeIdType string

type VertexValue struct {
    SourceOfTruth VertexIdType
    Partition     int
    Neighbors     map[VertexIdType]bool
}
```

```
9
10 type EdgeValue struct {
11 }
12
13 type PregelMessage struct {
14     SourceOfTruth   VertexIdType
15     SenderPartition int
16 }
```

LISTING A.6 – Go-Pregel methods for the Graph Bipartition algorithm

```
1  func (vertex *Vertex) ComputeInSuperStepZero() {
2      neighbors := make(map[VertexIdType]bool)
3      for _, edge := range vertex.GetOutEdges() {
4          neighbors[edge.To] = true
5
6          // Send message to neighbors
7          vertex.PrepareMessageToVertex(
8              edge.To,
9              PregelMessage{
10                 SourceOfTruth:   vertex.Id,
11                 SenderPartition: 0,
12             },
13         )
14     }
15     vertex.SetValue(
16         VertexValue{
17             SourceOfTruth: vertex.Id,
18             Partition:     0,
19             Neighbors:     neighbors,
20         },
21     )
22 }
23
24 func (vertex *Vertex) Compute(receivedMessages []PregelMessage) {
25     vertexValue := vertex.GetValue()
26     neighbors := vertexValue.Neighbors
27     sourceOfTruth := vertexValue.SourceOfTruth
28     partition := vertexValue.Partition
29     hasChanged := false
30
31     for _, message := range receivedMessages {
32         if vertex.GetSuperStepNumber() == 1 {
33             neighbors[message.SourceOfTruth] = true
34         }
```

```
        if partition == -1 {
            if message.SenderPartition != -1 {
                hasChanged = true
            }
            continue
        }

        if message.SenderPartition == -1 {
            hasChanged = true
            sourceOfTruth = message.SourceOfTruth
            partition = -1
            continue
        }

        if message.SourceOfTruth < sourceOfTruth {
            hasChanged = true
            sourceOfTruth = message.SourceOfTruth
            partition = 1 - message.SenderPartition
            continue
        }

        if message.SourceOfTruth == sourceOfTruth && message.SenderPartition ==
        ↪  partition {
            hasChanged = true
            partition = -1
        }
    }

    vertex.SetValue(
        VertexValue{
            SourceOfTruth: sourceOfTruth,
            Partition:     partition,
            Neighbors:     neighbors,
        },
    )

    if hasChanged || vertex.GetSuperStepNumber() == 1 {
        for neighbor := range neighbors {
            vertex.PrepareMessageToVertex(
                neighbor,
                PregelMessage{
                    SourceOfTruth:   sourceOfTruth,
                    SenderPartition: partition,
                },
            )
        }
```

```
80      } else {
81          vertex.VoteToHalt()
82      }
83  }
```

FOLHA DE REGISTRO DO DOCUMENTO

1. CLASSIFICAÇÃO/TIPO	2. DATA	3. REGISTRO N°	4. N° DE PÁGINAS
TC	27 de novembro de 2024	DCTA/ITA/TC-138/2024	78

5. TÍTULO E SUBTÍTULO:

Exploring Distributed Graph Processing Techniques

6. AUTOR(ES):

Gabriel Gandour

7. INSTITUIÇÃO(ÕES)/ÓRGÃO(S) INTERNO(S)/DIVISÃO(ÕES):

Instituto Tecnológico de Aeronáutica - ITA

8. PALAVRAS-CHAVE SUGERIDAS PELO AUTOR:

Graphs, Distributed Systems, Pregel

9.PALAVRAS-CHAVE RESULTANTES DE INDEXAÇÃO:

1. Processamento distribuído 2. Gráficos (computação) 3. Computação

10. APRESENTAÇÃO: **(X) Nacional ( ) Internacional**

ITA, São José dos Campos. Curso de Graduação em Engenharia de Computação. Orientadora: Profa. Dra. Juliana de Melo Bezerra. Publicado em 2024.

11. RESUMO:

After the development of graph theory, graphs have been used to study and solve problems in many fields, such as linguistics, biology and computer science. At the same time, the amount of data gathered by companies and institutions has reached a surprisingly high level. These two facts have led to the problem of processing and analysing large graphs in a reasonable time by using distributed algorithms. The solution to write distributed algorithms for graphs in a simple and consistent way was to build tools and frameworks that simplify the process of writing and running these algorithms, such as Google's Pregel, which is a distributed graph processing framework that has been used internally by Google to solve graph problems such as page rank. This work aims to study and understand the working principles of Pregel and, inspired by it, to propose, design and implement a new framework, called Go-Pregel. Go-Pregel focuses on the educational environment, helping students to understand distributed processing concepts and learn how to solve graph problems in a distributed environment. After explaining how Go-Pregel works, we give different graph problems examples and how to solve them using this framework. Finally, we conducted a practical activity with students from a distributed processing course, and present the collected results and feedback.

12. GRAU DE SIGILO:

(X) OSTENSIVO ( ) RESERVADO ( ) SECRETO